



"Comparative Evaluation of Centralized and Distributed Control Architectures for Web Traffic Anomaly Detection using Machine Learning and Deep Learning"

Palla Naveen Kumar Varma
3149417

Submitted in partial fulfilment for the degree of
Master of Science in Big Data Management and Analytics
Griffith College
MAY, 2026

Under the supervision of **Prof AQEEL KAZMI**

Disclaimer

I hereby certify that this material, which I now submit for assessment on the programme of study leading to the Degree of Master of Science in Big Data Management and Analytics at Griffith College Dublin, is entirely my own work and has not been submitted for assessment for an academic purpose at this or any other academic institution other than in partial fulfilment of the requirements of that stated above.

Signed:**Date:**

Acknowledgements

I would like to express my sincere gratitude to my supervisor **Professor Aqeel Kazmi**, for their invaluable guidance, encouragement, and continuous support throughout the completion of this thesis.

I also wish to thank the faculty members of the Department of Computer Science for providing the academic foundation and technical knowledge necessary for this research.

Special appreciation is extended to my family and friends for their unwavering encouragement, patience, and support throughout my academic journey.

Finally, I acknowledge all researchers, authors, and organizations whose prior work contributed significantly to the successful completion of this study.

List of Abbreviations

ANN	Artificial Neural Network
API	Application Programming Interface
DDoS	Distributed Denial of Service
DL	Deep Learning
GRU	Gated Recurrent Unit
HDFS	Hadoop Distributed File System
IoT	Internet of Things
LSTM	Long Short-Term Memory
ML	Machine Learning
PySpark	Python for Apache Spark
RF	Random Forest
RNN	Recurrent Neural Network
XAI	Explainable Artificial Intelligence

Contents

Acknowledgements	2
List of Abbreviations	3
Abstract	9
1 Introduction	10
1.1 Web Traffic Anomaly Detection and Control Architecture	10
1.2 Goals	11
1.3 Overview of Approach	11
1.3.1 Centralized Architecture	11
1.3.2 Distributed Architecture	12
1.4 Document Layout	13
2 Literature Review	14
2.1 Introduction	14
2.2 Web Traffic Anomaly Detection	14
2.3 Centralized Control Architectures	15
2.4 Distributed Control Architectures	15
2.5 Big Data Technologies	15
2.5.1 Hadoop Distributed File System (HDFS)	16
2.5.2 Apache PySpark	16
2.6 Machine Learning Approaches	16
2.7 Deep Learning Approaches	16
2.7.1 Artificial Neural Networks (ANN)	16
2.7.2 Gated Recurrent Units (GRU)	17
2.8 Emerging Deep Learning and Intelligent Anomaly Detection Paradigms	17
2.9 Comparative Research Studies	18
2.10 Research Gap	18
2.11 Chapter Summary	18
3 Methodology	20
3.1 Introduction	20
3.2 Research Framework	20
3.3 Dataset Acquisition	21
3.4 Distributed Storage Using Hadoop HDFS	21
3.5 Data Preprocessing Using Apache PySpark	22
3.6 Feature Engineering	22

3.7	Threshold Modeling	23
3.8	Machine Learning Model Methodology	23
3.9	Deep Learning Model Methodology	23
3.10	Evaluation Metrics	24
3.11	Comparative Analysis Framework	24
3.12	Chapter Summary	24
4	System Analysis and Design	25
4.1	Introduction	25
4.2	System Requirements Analysis	25
4.3	System Design Principles	26
4.4	Overall System Architecture	26
4.5	Centralized Architecture Design	27
4.6	Distributed Architecture Design	28
4.7	Software Framework Selection	29
4.8	Workflow Design	29
4.9	Comparative Design Considerations	29
4.10	Design Constraints and Risk Considerations	30
4.11	Enterprise Scalability and Security Considerations	30
4.12	Chapter Summary	30
5	System Implementation	31
5.1	Introduction	31
5.2	Development Environment	31
5.3	Hadoop HDFS Implementation	31
5.4	Apache PySpark Pipeline Implementation	32
5.5	Feature Engineering Implementation	34
5.6	Threshold Classification Implementation	35
5.7	Centralized Architecture Implementation	36
5.8	Distributed Architecture Implementation	36
5.8.1	Unified Predictive Model Input Preparation	36
5.8.2	Logistic Regression	37
5.8.3	Decision Tree	38
5.8.4	Random Forest	39
5.9	Deep Learning Model Implementation	40
5.9.1	Artificial Neural Networks (ANN)	40
5.9.2	Gated Recurrent Units (GRU)	42
5.10	Hyperparameter Tuning	43
5.11	Implementation Challenges	44
5.12	Chapter Summary	44
6	Testing, Results, and Comparative Evaluation	45
6.1	Introduction	45
6.2	Testing Strategy	45
6.3	Evaluation Metrics	46
6.4	System-Level Evaluation Metrics	47

6.4.1	Compression Ratio	47
6.4.2	Variability Ratio	47
6.4.3	Entropy Analysis	47
6.4.4	Information Loss	48
6.4.5	Trend Distortion	48
6.4.6	Availability and Reliability	48
6.5	Centralized Architecture Performance	48
6.5.1	Compression Efficiency and Data Reduction	49
6.5.2	Information Loss	50
6.6	Distributed Architecture Performance	51
6.6.1	Deep Learning Scalability Analysis	52
6.7	Comparative Model Performance	53
6.7.1	Predictive Performance Comparison Across Architectures	54
6.7.2	Behavioral Preservation Comparison	54
6.7.3	Compression and Information Trade-Off	55
6.7.4	Operational Availability and Reliability Comparison	55
6.7.5	Overall Comparative Findings	56
6.8	Discussion of Findings	57
6.9	Chapter Summary	57
7	Conclusion and Future Work	58
7.1	Introduction	58
7.2	Research Summary and Final Conclusions	58
7.3	Major Contributions of the Research	59
7.4	Research Limitations	59
7.5	Future Work Recommendations	60
7.6	Final Reflection	61

List of Figures

3.1	Hadoop architecture	21
4.1	Overall system architecture	27
4.2	Centralized architecture workflow	28
4.3	Distributed architecture workflow	28
5.1	Hadoop Architecture	32
5.2	Apache PySpark Distributed Data Preprocessing Pipeline	33
5.3	Decision Tree	39
5.4	Random Forest	40
5.5	Artificial Neural Network	41
5.6	GRU	43
6.1	Confusion Matrix Framework for Web Traffic Anomaly Detection	46
6.2	Predictive performance comparison across all centralized models.	49
6.3	Compression ratio comparison illustrating substantial data reduction under centralized aggregation.	50
6.4	Predictive performance comparison across distributed machine learning and averaged deep learning models.	51
6.5	ANN and GRU predictive scalability across increasing distributed dataset sizes. .	53
6.6	Predictive performance comparison between centralized and distributed architectures across core models.	54
6.7	Behavioral preservation comparison across centralized and distributed architectures.	55
6.8	Operational availability and reliability comparison between architectures.	56

List of Tables

6.1	Centralized Dataset Performance Comparison Across All Machine Learning and Deep Learning Models	49
6.2	Centralized Compression Ratio Evaluation	49
6.3	Information Loss Summary	50
6.4	Distributed Dataset Performance Comparison Across All Machine Learning and Deep Learning Models	51
6.5	Distributed Deep Learning Scalability Across Multiple Dataset Sizes	52
6.6	Centralized vs Distributed Predictive Performance Comparison Across Core Models	54
6.7	Behavioral Preservation Comparison Across Architectures	55
6.8	Centralized Compression and Information Loss	55
6.9	Availability and Reliability Comparison Across Architectures	56

Abstract

Modern web infrastructures generate massive and highly dynamic traffic volumes, creating significant challenges for scalable and resilient anomaly detection. Traditional centralized control architectures offer computational simplicity and strong global monitoring but often suffer from limited scalability, single points of failure, and reduced sensitivity to localized traffic anomalies. Distributed architectures, by contrast, preserve local behavioral intelligence, improve resilience, and better support large-scale environments.

In this thesis, I compare centralized and distributed architectures for web traffic anomaly detection head-to-head. I used HDFS, PySpark, and both machine learning and deep learning models. For features, I relied on statistical engineering and adaptive threshold modeling. The predictive models I tested include Logistic Regression (LR), Decision Trees (DT), Random Forest, Artificial Neural Networks (ANN), and Gated Recurrent Units (GRU).

Experimental findings demonstrate that centralized architectures provide superior computational efficiency and strong traditional machine learning performance, while distributed architectures significantly outperform in scalability, fault tolerance, behavioral preservation, and localized anomaly sensitivity. GRU-based distributed systems emerged as the strongest long-term solution due to their ability to preserve sequential traffic intelligence.

This research confirms that system architecture is equally as important as predictive model sophistication in determining effective anomaly detection performance. The findings provide valuable academic and practical contributions toward future resilient enterprise-scale web traffic monitoring systems.

Chapter 1

Introduction

1.1 Web Traffic Anomaly Detection and Control Architecture

The Internet is becoming more complex as technology continues to grow and evolve, alongside cloud computing platforms and large-scale digital infrastructures, making it increasingly difficult to monitor and manage Web traffic systems. In today’s world, where organizations support millions of users, system reliability, availability, and performance are critical operational priorities. Abnormal traffic events such as cyberattacks, network failures, and unexpected traffic bursts are significant issues in digital ecosystems and pose substantial challenges for resilient services [1].

Web traffic anomaly detection is an integral part of big data analytics and distributed computing, aiming to detect unusual traffic patterns that may indicate system stress, malicious attacks, or operational failure. Effective anomaly detection is essential because anomalies may serve as early warning signs of service degradation, Distributed Denial of Service (DDoS) attacks, server failures, or resource bottlenecks [18]. Traditional traffic monitoring systems typically utilize centralized control architectures in which all traffic data is collected and processed by a global controller based on fixed thresholds or predetermined policies [3].

While centralized architectures may be easier to design and maintain, they also have significant drawbacks. They contain single points of failure, lack scalability, are less adaptable to page-specific behaviors, and may create bottlenecks in large-scale systems [30]. Flaws in centralized systems have been observed in practice, demonstrating that relying on one central point of decision-making in dynamic and heterogeneous environments can be highly risky [9].

Distributed or resilient control architectures provide an alternative in which responsibilities for anomaly detection are decentralized. Distributed systems determine local traffic behavior and adapt thresholds according to historical traffic patterns for each page or node [1]. This can improve scalability, preserve local traffic behavior, increase fault tolerance, and reduce dependency on central processors [3]. With the increasing adoption of edge computing and resilient service design in modern infrastructures, distributed anomaly detection frameworks have gained significant attention [9].

This study looks at which architecture performs better in terms of reliability, scalability, detection accuracy, and how well it preserves local traffic behavior. The comparison is done using HDFS, Apache Spark, and a few advanced classification models [14].

1.2 Goals

The main goal of this project is pretty straightforward: design, build, and compare centralized and distributed control architectures for spotting anomalies in large-scale web traffic. I'm using modern tools from distributed systems, big data analytics, machine learning, and deep learning to put together a working detection framework. The real aim is to see which architecture does better—centralized or distributed—by looking at reliability, scalability, and resilience, while also noting where each one falls short.

Specific objectives include:

- Build scalable data pipelines that can handle distributed processing
- Pull out statistical features that capture normal traffic behavior
- Create centralized models using global thresholds
- Develop distributed models that adapt thresholds page-by-page
- Test classic ML models: Logistic Regression, Decision Trees, and Random Forest
- Test DL Models: Artificial Neural Networks(ANN) and Gated Recurrent Networks(GRU)

This research benefits academic researchers, system engineers, cloud architects, and cybersecurity professionals seeking resilient infrastructure solutions [29].

1.3 Overview of Approach

To accomplish these goals, a structured development lifecycle was adopted.

First, a large-scale web traffic time-series dataset was acquired and stored in Hadoop HDFS to support distributed storage and scalable processing [15].

Apache PySpark was then used for:

- Data cleaning
- Missing value handling
- Duplicate removal
- Data transformation
- Feature engineering

Key engineered features include:

- Moving averages
- Moving standard deviation
- Lag variables
- Trend analysis
- Deviation
- Z-score

Two architectural frameworks were implemented:

1.3.1 Centralized Architecture

In the centralized dataset, the number of traffic counts for each day, regardless of which page they occurred on, is used for each record, treating the whole system as a single analytical entity. Adaptive thresholds compute thresholds based on overall traffic behavior from the complete dataset, allowing anomaly detection based on global trends in system-wide traffic rather than page-specific deviations. This centralized view provides a comprehensive perspective on web traffic system performance and can be useful for detecting large-scale failures or broad

performance degradation across the entire infrastructure [2].

1.3.2 Distributed Architecture

In the distributed dataset, each record represents a unique page identifier and date, enabling evaluation of system state for each page-time combination. Local historical traffic pattern-based adaptive thresholds allow anomaly detection to focus on page-specific anomalies rather than overall site-wide traffic behavior. This granular approach improves sensitivity to localized anomalies and supports distributed analysis across large-scale web traffic volumes [1].

Machine learning and deep learning models are then trained on traffic classifications as follows:

- **Core Predictive Performance Metrics**
 - **Accuracy:** Indicates the overall prediction capability of the model by showing how many traffic instances were correctly classified out of the total observations.
 - **Precision:** Reflects how accurately the model identifies anomalous traffic by measuring the number of predicted anomalies that are actually anomalous.
 - **Recall:** Describes the ability of the model to detect anomalous events from all existing anomalies within the dataset.
 - **F1-score:** A single metric that balances precision and recall, useful when you don't want to optimize one at the expense of the other.
- **System Architecture Metrics**
 - **Information Loss:** Measures the amount of original data detail lost during distributed transformation, compression, or preprocessing.
 - **Compression Ratio:** Evaluates the efficiency of data size reduction after preprocessing or distributed storage optimization.
 - **Trend Distortion:** Assesses the extent to which original temporal traffic patterns are altered during data transformation or compression.
 - **Scalability:** Evaluates the system's ability to maintain performance efficiency as dataset size or computational load increases.
- **Data Behavioral Metrics**
 - **Entropy:** Quantifies the uncertainty or randomness within system state classifications, reflecting data complexity.

This methodology enables comprehensive technical implementation while supporting critical analytical comparison [17].

1.4 Document Layout

The remaining chapters of this thesis are organized as follows:

- **Chapter 2** covers background and related work.
- **Chapter 3** explains my methodology (data, features, thresholds, system dev).
- **Chapter 4** discusses analysis, requirements, tools, and architecture.
- **Chapter 5** walks through the implementation process.
- **Chapter 6** focuses on testing, evaluation, and performance results.
- **Chapter 7** concludes with findings, limitations, and future work.
- **Bibliography** lists all references used.

Chapter 2

Literature Review

2.1 Introduction

Digital technologies are changing fast, and organizations are increasingly relying on cloud services, distributed platforms, and big computing infrastructures to get their work done. This reliance comes with a side effect—massive volumes of web traffic data. So figuring out when traffic behavior looks abnormal has become essential for keeping systems performant and reliable. Why does traffic go weird? Lots of reasons: DDoS attacks, malicious activity, unexpected surges, server failures, or infrastructure glitches.

Traditional traffic monitoring approaches commonly follow a centralized structure in which data from different sources is collected and processed through a single management point. While such systems provide easier administration and centralized control, they may encounter difficulties when operating in large-scale environments. As the amount of data and processing requirements increase, these systems can experience scalability constraints, performance bottlenecks, and risks associated with single points of failure.[1].

In traditional traffic monitoring systems, traffic data from various sources is collected and processed by a single global controller, usually located in a centralized data center. While these centralized systems offer easier implementation and unified control, they can be limited in scalability, create bottlenecks, and have single points of failure [30]. This becomes increasingly problematic in large-scale applications where millions of users access web services simultaneously.

Distributed control architectures have thus become viable alternatives to overcome these challenges. Distributed systems introduce a decentralized approach to traffic monitoring so that adaptive thresholding and page-specific anomaly detection can be performed locally. These architectures provide better scalability, fault tolerance, and maintenance of local behavioral patterns [15].

This chapter presents an overview of previous research concerning web traffic anomaly detection along with concepts related to centralized and distributed systems, big data frameworks, machine learning, deep learning, and comparative analyses.

2.2 Web Traffic Anomaly Detection

Web traffic anomaly detection focuses on identifying abnormal traffic patterns that differ significantly from regular network activity. These irregular activities may occur due to cyberattacks, network failures, sudden increases in traffic, or unexpected user behavior [18]. Traditional

techniques such as statistical threshold monitoring, rule-based intrusion detection systems, time-series forecasting, and signature-based detection have been widely used to identify these abnormalities. However, complex web systems are generating larger amounts of traffic data, these approaches often struggle to adapt to changing traffic patterns and may produce a higher number of false alarms. This has increased the need for more flexible and scalable approaches for modern anomaly detection systems.

Intelligent anomaly detection systems using machine learning and distributed processing techniques have gained increasing focus in recent studies, offering improved detection accuracy and scalability [6].

2.3 Centralized Control Architectures

Centralized control architectures operate by collecting and processing all traffic information through a single control node or management system. This approach has been followed because of its simplicity, manageability, easy deploying in nature. A centralized structure provides a complete view of overall traffic behavior, making system monitoring and decision-making more straightforward while maintaining consistency across the network.

Despite these advantages, centralized architectures also have several limitations. Since the system depends heavily on a single control point, failures at that point can affect the entire system. In addition, centralized approaches may experience scalability issues and processing bottlenecks as traffic volume increases. They can also be less adaptable to page-specific behavior and offer limited flexibility in resource utilization, particularly in large and dynamic environments.

It has been demonstrated that centralized systems can perform well in terms of prediction accuracy, but are less appropriate for highly distributed, large-scale infrastructures [30].

2.4 Distributed Control Architectures

Distributed control architectures use a decentralized approach in which individual nodes or pages maintain their own adaptive thresholds for detecting anomalies. Instead of relying on a single central controller, monitoring and decision-making responsibilities are shared across multiple components of the system. Distributed control Architecture improves Adaptability and fault tolerance while decreasing dependence on a centralised processing unit. It also helps preserve local traffic behavior and increases system resilience, particularly in large and dynamic environments with varying traffic patterns.

Despite these benefits, distributed architectures can introduce certain challenges. Communication between multiple nodes may create additional overhead, and maintaining synchronization across the system can become more complex. The overall design and implementation process may also require greater effort, and differences in local processing can occasionally affect prediction consistency across the network.

Distributed architectures are increasingly integrated into edge computing and modern cloud-native infrastructures [9].

2.5 Big Data Technologies

Large-scale anomaly detection requires scalable storage and computational frameworks.

2.5.1 Hadoop Distributed File System (HDFS)

HDFS was built to support the storage , managing of large-scale datasets in distributed environments. It provides features such as distributed storage, block-level replication, fault tolerance, horizontal scalability, and high-throughput data access. These capabilities make HDFS particularly suitable for handling large web traffic time-series datasets used in anomaly detection systems, where reliability and scalability are important requirements [15].

2.5.2 Apache PySpark

Apache PySpark is a widely used distributed computing framework which supports efficient large-scale data processing. It is commonly applied in tasks such as data preprocessing, in-memory computation, feature engineering, scalable machine learning, and large-scale data transformation. Its ability to process data efficiently across distributed systems makes it valuable for building high-performance anomaly detection applications.

Spark’s MLlib and PySpark frameworks significantly improve processing efficiency for distributed machine learning workflows [14].

2.6 Machine Learning Approaches

Now-a-days Machine learning models are becoming important for intelligent anomaly detection.

Common models in prior research are :

- Logistic Regression(LR)
- Decision Trees(DT)
- Random Forest(RF)

The above are used frequently due to their balance of interpretability, predictive performance, and computational scalability [19].

These models offer:

- Automated pattern recognition
- Improved classification
- Statistical learning capabilities
- Enhanced anomaly detection precision

However, model performance often depends heavily on feature engineering quality and system architecture [20].

2.7 Deep Learning Approaches

Deep learning models provide advanced representation learning capabilities [22].

2.7.1 Artificial Neural Networks (ANN)

ANN models are effective for:

- Complex nonlinear classification
- Large feature spaces
- Robust predictive modeling

2.7.2 Gated Recurrent Units (GRU)

GRU models are especially suitable for:

- Sequential traffic analysis
- Temporal anomaly detection
- Long-term dependency modeling
- Efficient recurrent computation

GRU networks are computationally lighter than LSTM while retaining strong temporal learning performance [25].

2.8 Emerging Deep Learning and Intelligent Anomaly Detection Paradigms

As research in anomaly detection continues to progress, increasing attention is being given to advanced deep learning techniques that extend beyond conventional ANN and GRU approaches. Emerging technologies includes Long Short-Term Memory (LSTM) networks[26], Transformer-related models, federated learning frameworks, and explainable artificial intelligence (XAI) have gained importance in addressing the challenges of large-scale web traffic anomaly detection systems.

LSTM models performs well in capturing the temporal dependencies in time series data . Their memory cell structure allows them to preserve important historical information over extended periods, making them highly suitable for identifying evolving traffic behaviors and long-duration anomalies [26]. Compared with GRU models, LSTMs often provide stronger long-range sequence retention due to their more complex gating mechanisms. However, this additional complexity generally results in more computational costs, more training times, and increase in resource requirements. In distributed environments where computational efficiency is essential, GRU often offers a more practical balance between predictive capability and system performance.

Transformer architectures represent another major development in modern anomaly detection research. Unlike some models like GRU or LSTM, Transformers use attention mechanisms that allow the system to evaluate relationships across entire traffic sequences simultaneously rather than processing data step by step [27]. This enables stronger performance in highly complex, high-dimensional environments where long-range dependencies are critical. Transformer-based systems have shown promising results in advanced network security and anomaly detection tasks, particularly when dealing with large-scale or highly dynamic traffic patterns. Despite these advantages, Transformers generally require significantly more computational power and larger datasets, which may limit their practicality for smaller or resource-constrained distributed systems.

Federated learning has also evolved as a highly important innovation in distributed anomaly detection. Rather than sending all traffic data to a central location, federated learning allows local nodes or devices to train models independently while sharing only learned parameters with a global system [4]. This decentralized approach improves privacy, reduces centralized bottlenecks, and supports scalable deployment across geographically distributed infrastructures. Federated learning is especially valuable in edge computing, IoT ecosystems, and cloud-native environments where privacy preservation and decentralized intelligence are increasingly important.

In addition to predictive capability, explainability has become a growing priority in intelligent anomaly detection systems. Explainable artificial intelligence (XAI) frameworks aim to improve transparency by helping system operators understand why anomalies are detected rather than relying solely on black-box predictions [28]. Techniques such as SHAP values, feature importance analysis, and interpretable model frameworks improve operational trust, support regulatory compliance, and strengthen enterprise decision-making. This is particularly important in cybersecurity and infrastructure management, where understanding system behavior can be as valuable as detection accuracy itself.

Overall, these emerging technologies indicate that future anomaly detection systems will likely move toward architectures that combine distributed intelligence, scalable deep learning, privacy preservation, and explainability. While traditional models such as Random Forest, ANN, and GRU remain highly effective, these advanced paradigms provide valuable directions for future system development as digital infrastructures continue to grow in complexity.

2.9 Comparative Research Studies

Several prior studies have examined:

- Distributed anomaly detection frameworks
- Spark-based machine learning systems
- Edge computing architectures
- Deep learning-based traffic monitoring
- Centralized system limitations

Marfo et al. proposed distributed edge anomaly detection frameworks emphasizing resilient decentralized architectures, while Ahmed demonstrated scalable Spark-based anomaly detection pipelines. These studies reinforce the importance of architecture-level scalability in anomaly detection systems [1, 2].

Although many studies address isolated components, relatively few provide comprehensive comparisons between centralized and distributed architectures using both machine learning and deep learning frameworks [1].

2.10 Research Gap

Existing literature reveals several limitations:

- Limited end-to-end architecture comparisons
- Insufficient evaluation of behavioral preservation metrics
- Few studies integrating HDFS, Spark, ML, and DL simultaneously
- Limited analysis of GRU performance in distributed systems
- Incomplete exploration of scalability versus predictive accuracy trade-offs

This thesis addresses these gaps by developing and evaluating a full comparative framework for centralized and distributed web traffic anomaly detection architectures.

2.11 Chapter Summary

This literature review demonstrates that while significant progress has been made in anomaly detection, scalability, and intelligent classification, substantial gaps remain in architecture-level

comparative research. By integrating distributed storage, big-data processing, ML, and DL, this research contributes a comprehensive evaluation of modern web traffic anomaly detection systems.

Chapter 3

Methodology

3.1 Introduction

This chapter walks through the methodological framework used to design, build, and evaluate the web traffic anomaly detection system—under both centralized and distributed architectures. The methodology was developed to ensure that the research followed a structured and technically rigorous process capable of supporting large-scale data processing, predictive analysis, and system-level comparison [1].

The methodology combines several major stages, beginning with large-scale dataset acquisition and distributed storage, followed by data preprocessing, feature engineering, adaptive threshold modeling, predictive model development, final comparative evaluation. Each stage was designed to progressively transform raw web traffic data into meaningful analytical outputs while preserving important behavioral characteristics required for accurate anomaly detection [2].

This chapter aims to establish a structured and scalable framework that can assess predictive performance, system resilience, scalability, and behavioral preservation in modern web traffic environments. Since large-scale digital infrastructures generate massive and highly dynamic traffic data, this methodological framework was specifically designed to handle complexity while maintaining analytical reliability [9].

3.2 Research Framework

A structured research and development lifecycle was adopted to guide the implementation of the proposed system from initial data acquisition to final comparative analysis. This framework ensured consistency throughout the research process and allowed each stage of development to build logically upon the previous one.

The research process began with selecting an appropriate large-scale web traffic dataset capable of representing realistic system behavior over time. Once acquired, the dataset was stored within a distributed environment using Hadoop HDFS to provide scalable storage and computational support [15]. Following this, Apache PySpark was used to preprocess the data, perform cleaning operations, restructure the dataset, and engineer predictive features [14].

After preprocessing, adaptive threshold generation techniques were applied to classify traffic behavior into operational states. Machine learning and deep learning models were then developed using engineered features and generated classifications. Finally, all systems were evaluated using predictive, system, and behavioral metrics to support comprehensive comparative analysis.

This structured framework ensured that the research remained systematic, reproducible, and analytically sound.

3.3 Dataset Acquisition

A large-scale web traffic time-series dataset was selected to provide realistic traffic behavior suitable for anomaly detection experimentation. The chosen dataset consisted of thousands of unique web pages monitored over approximately 550 days, providing both scale and temporal continuity.

This dataset was particularly valuable because it captured daily traffic observations across multiple pages, thereby reflecting the natural complexity of modern web ecosystems. It included page identifiers, historical traffic counts, and temporal continuity across multiple years, making it suitable for evaluating both localized anomalies and large-scale traffic disruptions.

The use of a large multivariate dataset was essential for this research because smaller or less complex datasets may fail to represent the challenges faced by real-world web traffic infrastructures. By using a sufficiently large and diverse dataset, the research was able to evaluate predictive performance under realistic large-scale conditions [18].

3.4 Distributed Storage Using Hadoop HDFS

To manage the large volume of traffic data efficiently, Hadoop Distributed File System (HDFS) was implemented as the primary storage infrastructure. HDFS was selected because of its proven ability to store massive datasets across distributed nodes while maintaining fault tolerance and scalability [15].

The implementation of HDFS allowed the dataset to be partitioned into distributed blocks, enabling parallel access and large-scale computational processing. Data replication mechanisms further improved resilience by ensuring that node failures would not result in critical data loss.

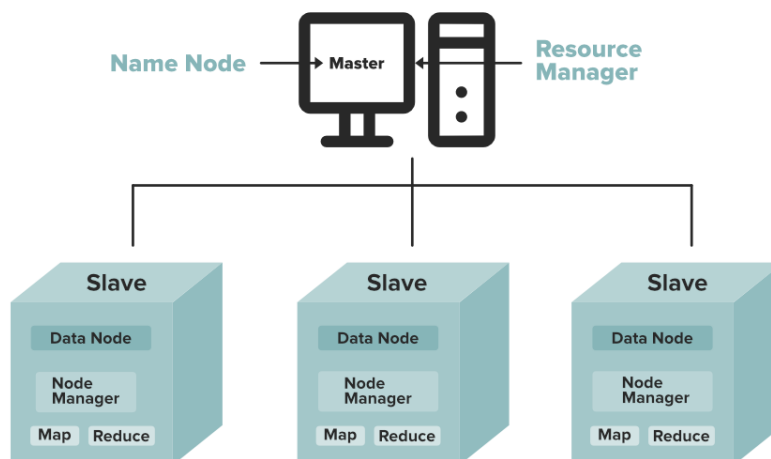


Figure 3.1: Hadoop architecture

This distributed storage architecture provided the foundational infrastructure for subsequent large-scale preprocessing and machine learning tasks. By integrating HDFS into the methodological framework, the research ensured that the system could realistically support enterprise-scale

traffic analytics while preserving reliability and scalability.

3.5 Data Preprocessing Using Apache PySpark

Apache PySpark served as the primary platform for distributed preprocessing due to its compatibility with HDFS, in-memory computing efficiency, and ability to process massive datasets rapidly across distributed environments [14].

Web traffic datasets are inherently complex and often contain missing values, duplicated records, inconsistent formats, noisy observations, and structural irregularities. These issues can significantly reduce predictive reliability if left untreated. Therefore, preprocessing was a critical methodological stage aimed at transforming raw traffic data into analytically reliable input for predictive modeling.

PySpark was used to clean corrupted records, address missing values, eliminate duplicates, standardize temporal formats, normalize traffic scales, and restructure raw data into machine learning-ready formats. In addition, PySpark was used to generate percentile classifications and prepare both centralized and distributed versions of the dataset.

The distributed nature of PySpark allowed these tasks to be executed efficiently across large-scale data volumes, significantly reducing computational overhead while improving preprocessing consistency. This stage ensured that all subsequent modeling processes were built on high-quality and behaviorally meaningful data [17].

3.6 Feature Engineering

Feature engineering was a critical methodological stage because raw traffic counts alone do not adequately capture the complex temporal, statistical, and behavioral properties required for effective anomaly detection.

To address this limitation, advanced rolling-window statistical methods were applied to generate predictive features capable of representing local traffic trends, volatility, deviations, and sequential dependencies. Rolling window analysis was particularly suitable because web traffic behavior evolves over time and generally includes both temporary variations and long-term behavioral patterns.

Several engineered features were developed, including moving averages, moving standard deviations, lag variables, trend indicators, deviation scores, and Z-scores. These features transformed raw traffic observations into multidimensional predictive variables that significantly improved model sensitivity to abnormal traffic behavior [19].

Window size optimization was also performed to balance responsiveness, noise reduction, and computational efficiency. This ensured that engineered features preserved meaningful traffic behavior while maximizing predictive relevance.

Overall, feature engineering acted as the bridge between raw traffic observations and predictive intelligence, enabling both machine learning and deep learning systems to identify anomalies more accurately.

3.7 Threshold Modeling

Threshold modeling was implemented to classify traffic observations into meaningful operational states based on adaptive percentile distributions. Traditional static thresholds are often ineffective in dynamic web environments due to continuous behavioral fluctuations across pages and time periods [18].

To overcome this limitation, adaptive percentile-based thresholding was employed. This approach dynamically adjusted thresholds according to historical traffic distributions, allowing classification systems to remain sensitive to genuine anomalies while reducing false positives.

Traffic was classified into three primary operational states: normal, degraded, and failure. In centralized architecture, thresholds were applied globally to aggregated system-wide traffic. In distributed architecture, thresholds were generated individually for each page based on local traffic behavior.

This methodology provided a balanced and flexible classification framework capable of preserving both global and localized traffic characteristics. Threshold modeling therefore served as the core decision-making layer that supported predictive system training.

3.8 Machine Learning Model Methodology

Machine learning techniques were utilized to perform scalable and interpretable prediction of traffic behavior. LR, DT, and RF models were chosen in terms of their individual strengths in areas such as interpretability, scalability, predictive capability, and suitability for distributed environments[21].

Logistic Regression served as a fundamental statistical model for classification tasks, while Decision Trees enabled the learning of nonlinear decision patterns. Random Forest further enhanced prediction performance by combining multiple decision trees through an ensemble-based approach. [21].

Logistic Regression provided a baseline statistical classifier, Decision Trees offered interpretable nonlinear decision boundaries, and Random Forest improved classification performance through ensemble learning.

These models were trained using engineered traffic features and threshold-generated traffic states. Their inclusion allowed the research to compare traditional machine learning effectiveness across both centralized and distributed architectures.

3.9 Deep Learning Model Methodology

To address complex nonlinear relationships and sequential patterns in web traffic behavior, deep learning techniques were incorporated into the study. Artificial Neural Networks (ANN) were applied to learn multidimensional feature relationships, whereas Gated Recurrent Units (GRU) were adopted due to their effectiveness in learning sequential patterns within time-series traffic data [22].

GRU models were particularly important because web traffic systems exhibit temporal continuity, making sequential learning highly valuable for anomaly detection [25].

The inclusion of deep learning models expanded the methodological framework beyond conventional classifiers, allowing the research to assess whether advanced neural architectures

provide superior predictive performance and behavioral preservation.

3.10 Evaluation Metrics

The proposed system was evaluated using predictive, architectural, and behavioral metrics to ensure multidimensional system assessment.

The proposed framework was assessed using predictive, system-level, and behavioral metrics to obtain a complete understanding of overall system performance.

Predictive measures including accuracy, precision, recall, and F1-score were used to evaluate the effectiveness of classification outcomes. System-related metrics such as scalability, compression ratio, information loss, and trend distortion were utilized to assess architectural efficiency and operational behavior. In addition, entropy was considered as a behavioral metric to examine the complexity and uncertainty present in traffic patterns [28].

This broader evaluation framework ensured that system performance was not judged solely on predictive capability, but also on infrastructure efficiency and behavioral consistency.

3.11 Comparative Analysis Framework

Comparative analysis was performed to evaluate differences between centralized and distributed architectures as well as between ML and DL models.

This comparison focused on predictive performance, computational scalability, resilience, behavioral preservation, and operational efficiency. This allowed the research to determine not only which predictive models performed best, but also which architectural strategies offered superior long-term suitability for large-scale web traffic environments [30].

By incorporating both technical performance and behavioral preservation into evaluation, the comparative framework provided a comprehensive basis for architecture-level decision-making.

3.12 Chapter Summary

This chapter discussed the overall methodological framework adopted for the development and evaluation of the proposed web traffic anomaly detection system. The methodology incorporated distributed storage mechanisms, scalable preprocessing techniques, feature engineering methods, adaptive threshold generation, predictive modeling approaches, and comparative evaluation strategies. Together, these components established the technical basis required for the subsequent stages of the research.

Overall, this chapter ensures that the system development process remains systematic, scalable, and academically rigorous while supporting meaningful comparison between centralized and distributed anomaly detection architectures.

Chapter 4

System Analysis and Design

4.1 Introduction

This chapter deals with the analysis and designed framework developed for the implementation of the web traffic anomaly detection platform [1]. The purpose of this chapter is to translate the methodological framework into a technically structured system capable of supporting large-scale traffic monitoring, predictive anomaly detection, and comparative architectural evaluation [30]. System analysis and design are essential stages in ensuring that the proposed framework is scalable, reliable, fault tolerant, and compatible with modern distributed computing environments [15].

The design of the system was guided by the need to process massive web traffic datasets efficiently while supporting both centralized and distributed anomaly detection architectures [17]. The system had to accommodate large-scale data ingestion, distributed storage, feature engineering, adaptive threshold generation, predictive model integration, and comparative performance analysis [2]. By carefully designing each layer of the system, the project ensures that the anomaly detection framework can function effectively in realistic enterprise-scale digital infrastructures [9].

4.2 System Requirements Analysis

The proposed anomaly detection system required both functional and non-functional capabilities to support its research objectives [6]. Functionally, the system needed to ingest and process large-scale web traffic datasets, perform distributed preprocessing, engineer predictive features, generate anomaly classification thresholds, train machine learning and deep learning models, and produce comparative analytical outputs [2]. These requirements were necessary to support end-to-end anomaly detection experimentation across both centralized and distributed architectures [3].

In addition to these operational capabilities, several non-functional requirements were identified. Scalability was a critical requirement because web traffic systems continuously grow in volume and complexity [14]. Fault tolerance was also essential to ensure that failures within distributed nodes would not compromise system reliability [15]. Processing efficiency, maintainability, modularity, and data consistency were equally important, as the system needed to support repeated experimentation, large-scale analytics, and reproducible research under enterprise-like conditions [30].

4.3 System Design Principles

The system design was guided by several core principles that ensured both technical robustness and academic validity [30]. Scalability was prioritized to ensure that the framework could process increasing traffic volumes without significant degradation in performance [14]. Fault tolerance was integrated through distributed storage and processing technologies to reduce system vulnerability to node failures [15]. Modularity was emphasized so that each system component, covering activities like data-preprocessing, feature extraction, predictive model development, performance assessment could operate independently while contributing to the larger framework [16].

Behavioral preservation was another important design principle, as the system needed to maintain meaningful traffic characteristics throughout preprocessing and distributed transformations [18]. This was particularly important for anomaly detection because excessive transformation could distort critical traffic patterns and reduce predictive effectiveness [7]. Finally, computational efficiency was prioritized to minimize resource consumption while maximizing analytical throughput, particularly when processing large-scale datasets [14].

4.4 Overall System Architecture

The overall system architecture was designed as a multi-layered framework integrating distributed storage, large-scale data processing, predictive intelligence, and comparative evaluation [17]. The architecture begins with data acquisition, where raw web traffic datasets are collected and prepared for distributed storage. The Hadoop Distributed File System (HDFS) was adopted as the primary storage component for handling traffic datasets within the distributed environment. Its use allows the system to efficiently manage increasing data volumes while maintaining reliability and fault tolerance during data storage and access operations [15].

Above the storage layer, Apache PySpark was implemented as the distributed preprocessing and feature engineering engine [14]. PySpark enables efficient cleaning, transformation, restructuring, and feature generation across large-scale traffic datasets [2]. Following preprocessing, adaptive threshold modeling is applied to classify traffic states into operational categories such as normal, degraded, and failure [18].

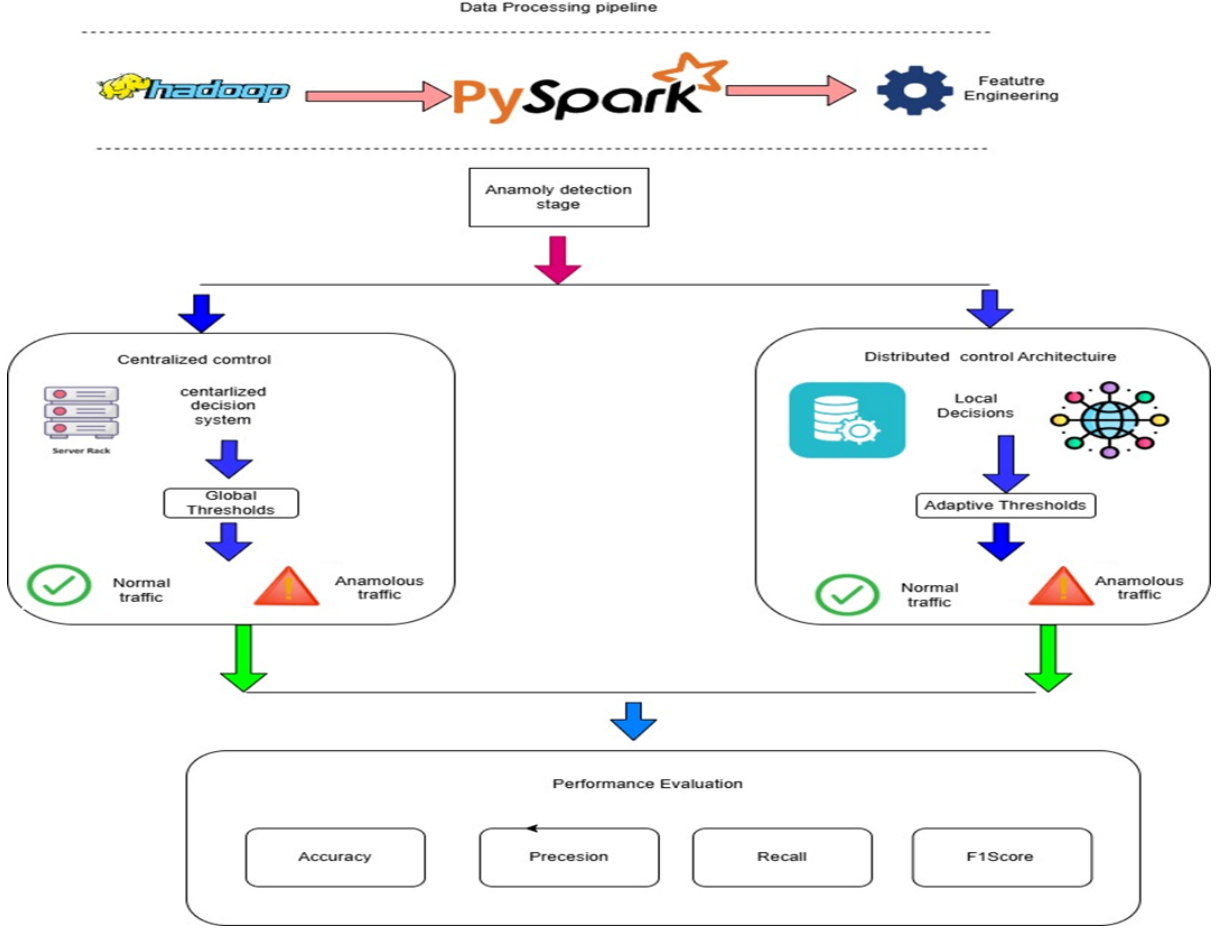


Figure 4.1: Overall system architecture

The architecture then branches into two primary pathways: centralized and distributed anomaly detection frameworks [1]. In the centralized model, aggregated traffic data is analyzed globally, while the distributed model preserves localized page-level traffic behavior [30]. Machine learning and deep learning models are subsequently trained on both architectures to evaluate predictive performance [25]. Finally, a comparative evaluation layer assesses predictive metrics, scalability, behavioral preservation, and system resilience [28].

4.5 Centralized Architecture Design

The centralized architecture was designed to treat the web traffic ecosystem as a unified analytical entity [2]. In this framework, traffic counts from all web pages are aggregated into daily system-wide observations. This aggregated approach enables global threshold generation and unified anomaly classification across the entire traffic system [18].

The primary advantage of centralized design lies in its simplicity and strong global visibility. Because all traffic behavior is processed through a single analytical framework, system-wide anomalies such as broad service degradation or large-scale cyberattacks can be detected efficiently [6]. Centralized systems also offer easier deployment and simplified management due to their unified structure [2].

However, this design introduces several limitations. Centralized architectures create potential bottlenecks due to the concentration of processing responsibilities [30]. They also represent single points of failure, reducing resilience under dynamic operational conditions [1]. Additionally,

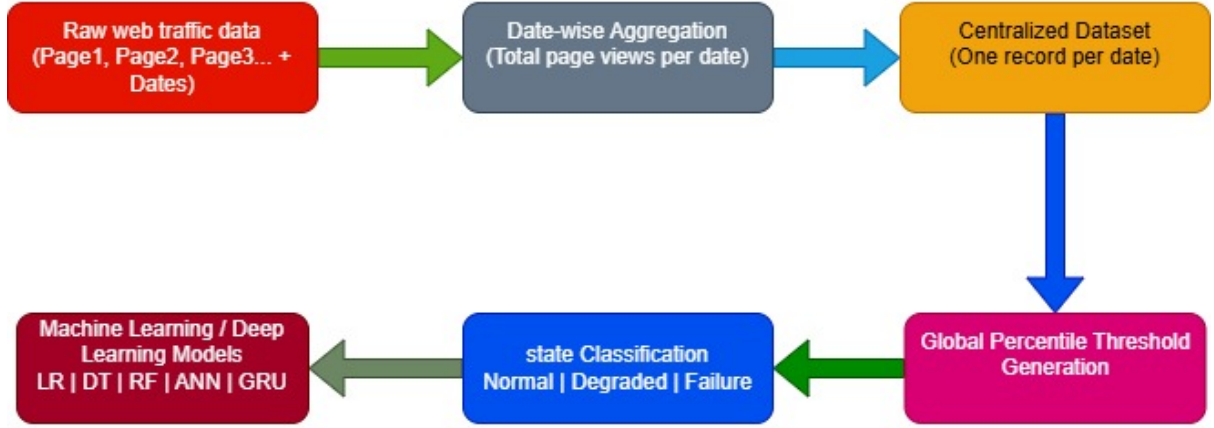


Figure 4.2: Centralized architecture workflow

global aggregation may suppress localized page-specific anomalies, reducing sensitivity to smaller-scale behavioral abnormalities [7]. These trade-offs make centralized systems suitable for broad visibility but less adaptable to highly distributed infrastructures [9].

4.6 Distributed Architecture Design

The distributed architecture was developed to decentralize anomaly detection by preserving local traffic intelligence across individual web pages [1]. Instead of aggregating all traffic globally, each page-date pair is treated as an independent analytical record. This design allows page-specific adaptive thresholds to be generated based on historical local traffic patterns [3].

Distributed design offers substantial scalability advantages because computational responsibilities can be distributed across multiple processing nodes [14]. This improves resilience, reduces centralized bottlenecks, and enhances sensitivity to localized anomalies [30]. Distributed architectures are particularly aligned with modern cloud-native and edge computing infrastructures, where decentralized intelligence is increasingly important [10].

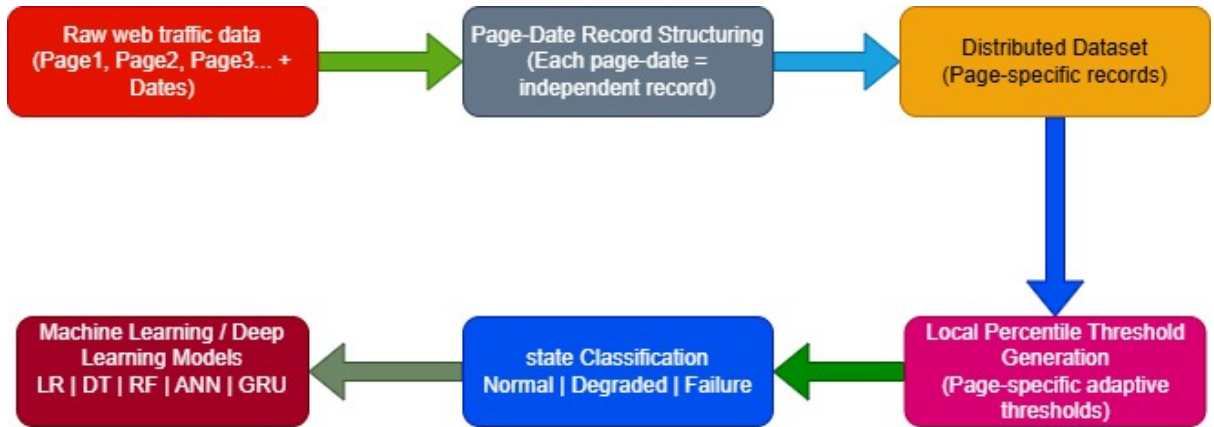


Figure 4.3: Distributed architecture workflow

Despite these advantages, distributed systems are inherently more complex to design and manage. Synchronization across distributed nodes can introduce communication overhead, and ensuring consistency across decentralized systems can be challenging [4]. Nevertheless, the distributed architecture prioritizes resilience, scalability, and local anomaly sensitivity, making it

highly relevant for modern large-scale traffic systems [9].

The distributed architecture treated each page-date pair as an independent analytical unit.

Key characteristics:

- Page-specific thresholds
- Localized anomaly preservation
- Improved scalability
- Greater resilience

4.7 Software Framework Selection

The selection of software frameworks was based on scalability, computational efficiency, predictive flexibility, and compatibility with distributed infrastructures [14]. Hadoop HDFS was chosen due to its distributed storage capabilities, replication mechanisms, fault tolerance, and ability to manage large-scale datasets effectively [15].

Apache Spark and PySpark were selected because of their in-memory distributed processing capabilities, which significantly improve preprocessing and feature engineering efficiency compared to traditional batch-processing frameworks [14]. Spark’s scalability and machine learning integration make it particularly suitable for enterprise-scale anomaly detection research [16].

For predictive modeling, Scikit-learn was used for machine learning implementations, while TensorFlow and Keras supported deep learning model development [21]. This combination of technologies provided a robust and flexible software ecosystem capable of supporting both traditional statistical learning and advanced neural architectures [22].

4.8 Workflow Design

The system workflow follows a sequential yet modular structure designed to maximize analytical consistency and scalability [2]. Raw traffic data is first acquired and stored in HDFS. PySpark then preprocesses the data, performs structural transformations, and generates predictive features [17]. Adaptive threshold modeling is subsequently applied to classify traffic states [18].

Following threshold classification, traffic data is restructured into centralized and distributed analytical formats. Machine learning and deep learning models are then trained independently on each architecture [25]. Finally, predictive, architectural, and behavioral metrics are evaluated to determine comparative system performance [28].

This modular workflow ensures reproducibility, simplifies debugging, and allows each stage to be independently optimized without compromising the integrity of the larger system [30].

4.9 Comparative Design Considerations

A key objective of the system design was to support fair and rigorous comparison between centralized and distributed architectures [1]. To achieve this, shared components such as dataset source, preprocessing procedures, feature engineering techniques, and predictive models were standardized across both architectures. Only the architectural control structures themselves differed.

This way, if I see a difference in performance, I know it’s because of the architecture, not

something else messing with the results [30]. So the comparison comes down to real trade-offs: predictive performance, scalability, resilience, preserving behavioral patterns, and computational efficiency [29].

4.10 Design Constraints and Risk Considerations

Several technical constraints influenced system design decisions. Computational resource limitations required efficient distributed processing frameworks [14]. Large-scale storage demands necessitated fault-tolerant distributed storage [15]. Synchronization challenges in distributed architectures required careful restructuring to preserve data consistency [4].

Potential risks included data distortion during preprocessing, resource bottlenecks in centralized systems, and communication overhead in distributed systems [7]. To mitigate these risks, the system incorporated HDFS replication, PySpark distributed processing, adaptive thresholding, and validation controls [2].

4.11 Enterprise Scalability and Security Considerations

Large-scale enterprise environments present challenges beyond those typically seen in controlled research settings. Modern organizations handle massive traffic volumes, dynamic user behavior, distributed services, and increasing cybersecurity threats. Therefore, anomaly detection systems must focus not only on predictive performance but also on scalability, security, resilience, and long-term operational efficiency.

Scalability was an important consideration because traffic demands can increase rapidly due to business growth or unexpected traffic surges. Distributed architectures support horizontal scaling by allowing additional processing nodes to be added as needed, improving flexibility and performance [14].

Security and reliability were also key factors. Unlike centralized systems, which depend on a single control point, distributed architectures reduce the impact of failures by distributing processes across multiple nodes, improving fault tolerance and resilience [30].

In addition, distributed systems support load balancing and integrate well with cloud-native technologies such as containers and orchestration platforms. These characteristics improve resource utilization and make distributed architectures more suitable for modern enterprise environments.

Overall, distributed architectures provide benefits beyond predictive accuracy by improving scalability, resilience, and long-term adaptability.

4.12 Chapter Summary

This chapter presented the system analysis and design framework for the proposed web traffic anomaly detection platform [1]. It defined the key requirements, architecture, design principles, software tools, and workflow needed to support scalable and reliable anomaly detection [30]. The design provides the foundation for the implementation and evaluation discussed in later chapters.

Chapter 5

System Implementation

5.1 Introduction

This chapter presents the practical implementation of the proposed web traffic anomaly detection system based on the methodologies and architectural designs established in previous chapters. The implementation phase focused on transforming the theoretical framework into a fully operational system capable of large-scale web traffic processing, anomaly classification, predictive intelligence, and comparative analysis.

The system was implemented through distributed storage deployment, scalable preprocessing pipelines, feature engineering, adaptive threshold classification, and the development of machine learning and deep learning models under both centralized and distributed architectures.

5.2 Development Environment

A hybrid development strategy was adopted to support distributed big data processing, machine learning, and deep learning experimentation.

Ubuntu Server virtual machines were used as the primary development platform for:

- Hadoop HDFS deployment
- Apache Spark preprocessing
- PySpark feature engineering
- Machine learning implementation
- ANN deployment

Google Colab was selectively used for GRU model development due to its GPU acceleration and improved computational performance for sequential deep learning tasks.

This hybrid strategy provided a practical balance between local distributed infrastructure and cloud-based deep learning resources.

5.3 Hadoop HDFS Implementation

Hadoop HDFS was implemented as the distributed storage foundation for managing large-scale web traffic datasets.

Its implementation ensured:

- Scalable distributed storage

- Fault tolerance
- High-throughput data access
- Reliable Spark integration

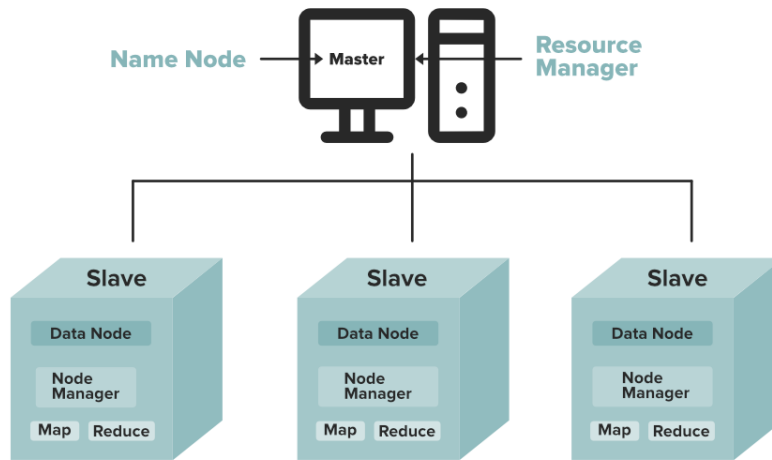


Figure 5.1: Hadoop Architecture

This storage infrastructure provided the foundational layer for distributed preprocessing and large-scale predictive analytics.

5.4 Apache PySpark Pipeline Implementation

Apache PySpark was implemented to develop the full distributed data preprocessing pipeline for large-scale web traffic analytics. The pipeline handled data ingestion, structural transformation, missing value treatment, normalization, feature engineering, threshold generation, and architecture-specific restructuring.

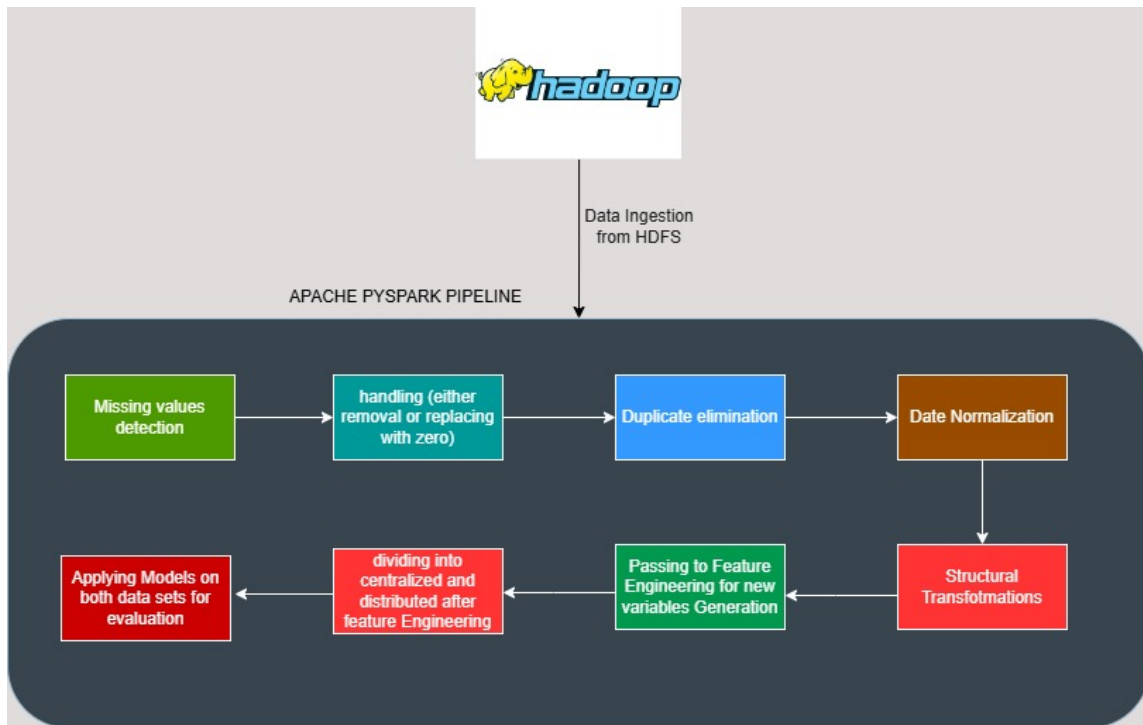


Figure 5.2: Apache PySpark Distributed Data Preprocessing Pipeline

```

1
2 # Load raw web traffic dataset
3 df = spark.read.csv(
4     "train_1.csv",
5     header=True,
6     inferSchema=True
7 )
8
9 # Transform wide-format data
10 pairs = array(*[
11     struct(
12         lit(c).alias("Date"),
13         col(c).alias("page_count")
14     )
15     for c in df.columns[1:]
16 ])
17
18 stacked_df = df.select(
19     "Page",
20     explode(pairs).alias("tmp")
21 ).select(
22     "Page",
23     "tmp.Date",
24     "tmp.page_count"
25 )
26
27 processed_df = stacked_df.fillna(
28     {"page_count":0}
29 ).withColumn(
30     "Date",
31     to_date("Date")

```

Listing 5.1: Data Preprocessing using PySpark

The Apache PySpark preprocessing pipeline provided a scalable and efficient foundation for processing large-scale web traffic data. By handling data ingestion, cleaning, normalization, feature generation, threshold modeling, and dataset restructuring, the pipeline transformed raw traffic records into reliable analytical datasets. This stage improved data quality and computational efficiency while preparing both centralized and distributed architectures for machine learning and deep learning model development.

5.5 Feature Engineering Implementation

Feature engineering was implemented using rolling statistical windows to transform raw traffic observations into predictive variables.

```

1
2  # Rolling-window feature engineering with optimized window size
3
4  window_lag = Window.partitionBy("Page").orderBy("Date")
5
6  window_opt = Window.partitionBy("Page") \
7                  .orderBy("Date") \
8                  .rowsBetween(-2,0)
9
10 final_df = FE_df.withColumn(
11     "Moving_avg",
12     round(avg("page_count").over(window_opt),2)
13 ).withColumn(
14     "Moving_std",
15     round(stddev("page_count").over(window_opt),2)
16 ).withColumn(
17     "lag_1",
18     lag("page_count",1).over(window_lag)
19 ).withColumn(
20     "trend",
21     col("page_count") - col("lag_1")
22 ).withColumn(
23     "deviation",
24     round(col("page_count")-col("Moving_avg"),2)
25 ).withColumn(
26     "z_score",
27     round(
28         col("deviation") /
29         col("Moving_std"),2
30     )
31 )

```

Listing 5.2: Rolling Window Feature Engineering Implementation

Generated features included:

- Moving averages
- Moving standard deviations
- Lag variables

- Trend indicators
- Deviation scores

These features significantly improved anomaly sensitivity and predictive quality.

5.6 Threshold Classification Implementation

Adaptive threshold modeling was implemented using percentile-based traffic classification.

```

1
2  # Adaptive threshold generation and state labeling
3
4  # Distributed percentile-based classification
5  window_dist = Window.partitionBy(
6      "Page"
7  ).orderBy("z_score")
8
9  df_distributed = df_active.withColumn(
10     "percentile",
11     percent_rank().over(
12         window_dist
13     )
14 ).withColumn(
15     "state_label",
16     when(
17         col("percentile") <= 0.40,
18         "Normal"
19     ).when(
20         col("percentile") <= 0.75,
21         "Degraded"
22     ).otherwise(
23         "Failure"
24     )
25 )
26
27 # Centralized global aggregation
28 df_centralized = df_active.groupBy(
29     "Date"
30 ).agg(
31     sum("page_count")
32     .alias("total_load")
33 )
34
35 # Global z-score calculation
36 df_centralized = df_centralized.withColumn(
37     "z_score",
38     (
39         col("total_load")
40         - global_mean
41     ) / global_stddev
42 )

```

Listing 5.3: Adaptive Threshold Generation and State Classification

Traffic states were dynamically labeled as:

- Normal

- Degraded
- Failure

This approach ensured architecture-specific state generation while preserving meaningful traffic behavior.

5.7 Centralized Architecture Implementation

The centralized implementation combined traffic data into daily system-wide records through global feature generation, threshold classification, and model training. This approach provided a complete view of overall system behavior; however, it was less effective in capturing smaller page-specific anomalies.

5.8 Distributed Architecture Implementation

The distributed implementation maintained page-level traffic information throughout preprocessing and predictive modeling, allowing the system to preserve local traffic behavior. Through page-specific feature generation and threshold modeling, anomalies could be detected independently at a more detailed level. This approach improved sensitivity to localized anomalies while supporting better behavioral preservation and scalability.

5.8.1 Unified Predictive Model Input Preparation

Before predictive model implementation, both centralized and distributed datasets underwent a shared machine learning preparation pipeline.

```

1
2  # Unified machine learning input pipeline
3  # for centralized and distributed datasets
4
5  feature_columns = [
6      "value",
7      "Moving_avg",
8      "Moving_std",
9      "lag_1",
10     "lag_2",
11     "trend",
12     "deviation"
13 ]
14
15 assembler = VectorAssembler(
16     inputCols=feature_columns, # These are my statical traffic features
17     outputCol="features"
18 )
19
20 assembled_df = assembler.transform(
21     final_df
22 )
23 # Standardize features to have zero mean and unit variance
24 # This helps models like LR and ANN converge faster
25 scaler = StandardScaler(
26     inputCol="features",

```

```

27     outputCol="scaled_features",
28     withMean=True, # center at zero (good for dense data)
29     withStd=True   #scale to unit variance
30 )
31
32 scaler_model = scaler.fit(
33     assembled_df
34 )
35
36 scaled_df = scaler_model.transform(
37     assembled_df
38 )
39 # 80% training and 20% testing
40 train_Thesis_data, test_Thesis_data = scaled_df.randomSplit(
41     [0.8,0.2],
42     seed=42 # Fixed seed ensures I get the same split every time I rerun
43 )

```

Listing 5.4: Unified Machine Learning Input Pipeline

This stage included:

- Feature vector assembly
- Standardization
- Label indexing
- Train-test splitting

This ensured fair and consistent training conditions across all machine learning models.

5.8.2 Logistic Regression

Logistic Regression was implemented as the baseline machine learning model because of its simplicity, efficiency, and interpretability [19]. It provided an initial benchmark for evaluating how effectively engineered traffic features could classify web traffic into normal, degraded, and failure states.

This model was particularly useful for identifying linear relationships between traffic patterns and anomaly states while supporting rapid large-scale training. Its low computational requirements made it practical for both centralized and distributed environments.

Although Logistic Regression offered reliable baseline performance, its linear nature limited its ability to capture more complex nonlinear traffic behaviors. Nevertheless, it served as an important reference point for comparing more advanced predictive models.

```

1
2 # Logistic Regression implementation
3 # using PySpark MLlib
4
5 from pyspark.ml.classification import \
6 LogisticRegression
7
8 # Initialize model
9 lr = LogisticRegression(
10     featuresCol="scaled_features", # from traffic features
11     labelCol="label", # anomaly label (1 for anomaly and 0 for normal)
12     maxIter=100
13 )

```

```

14
15
16 lr_model = lr.fit(
17     train_Thesis_data
18 )
19
20
21 lr_predictions = lr_model.transform(
22     test_Thesis_data
23 )

```

Listing 5.5: Logistic Regression Implementation

Although limited by linear assumptions, it served as an important benchmark for comparative analysis.

5.8.3 Decision Tree

Decision Tree was implemented to improve upon the linear limitations of Logistic Regression by introducing nonlinear classification capabilities [20]. Its hierarchical structure enabled the model to capture more complex traffic behaviors through feature-based decision rules.

This model provided greater flexibility while maintaining strong interpretability through clear branching logic. Decision Trees were particularly useful for understanding how specific traffic features contributed to anomaly detection under different architectures.

Although Decision Trees improved predictive sensitivity, they were more susceptible to overfitting when handling highly variable traffic data.

```

1
2  # Decision Tree implementation
3  # using PySpark MLlib
4
5  from pyspark.ml.classification import \
6      DecisionTreeClassifier
7
8  # Initialize model
9  Thesis_dt = DecisionTreeClassifier(
10     featuresCol="features",
11     labelCol="label"
12 )
13
14 # Train model
15 Theis_dt_model = Thesis_dt.fit(
16     train_Thesis_data
17 )
18
19 # Generate predictions
20 Thesis_dt_predictions = Thesis_dt_model.transform(
21     test_Thesis_data
22 )

```

Listing 5.6: Decision Tree Implementation

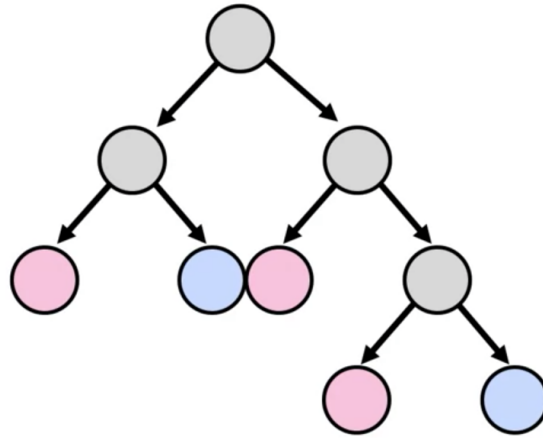


Figure 5.3: Decision Tree

It provided improved predictive flexibility but was more susceptible to overfitting.

5.8.4 Random Forest

Random Forest was implemented to improve predictive robustness, reduce overfitting, and enhance anomaly detection through ensemble learning [19]. By combining multiple decision trees, the model was able to capture complex nonlinear traffic behaviors more reliably.

This model was highly effective for handling dynamic, noisy, and multidimensional traffic environments, making it one of the strongest traditional machine learning models in this research.

Although Random Forest required greater computational resources and reduced interpretability compared to simpler models, its improved predictive strength made it a key benchmark for advanced anomaly detection.

```

1
2  # Random Forest implementation
3  # using PySpark MLlib
4
5  from pyspark.ml.classification import \
6  RandomForestClassifier
7
8  # Initialize model
9  my_random_forest = RandomForestClassifier(
10     featuresCol="features",      # my assembled feature vector
11     labelCol="label",           # anomaly flag (1 or 0)
12     numTrees=100                # enough trees for stable predictions
13
14  # Train model
15  train_forest = my_random_forest.fit(train_Thesis_data)
16
17
18  # Generate predictions
19  forest_predictions = train_forest.transform(test_Thesis_data)
20  )

```

Listing 5.7: Random Forest Implementation

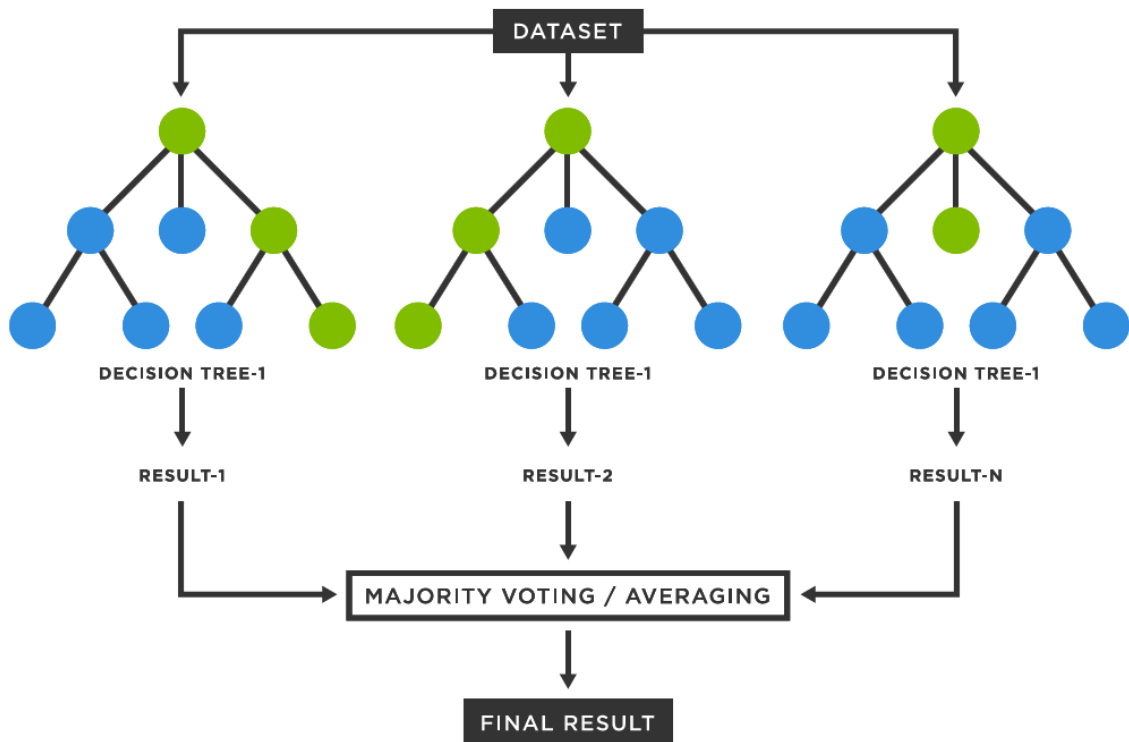


Figure 5.4: Random Forest

Random Forest provided one of the strongest traditional machine learning performances in this project.

5.9 Deep Learning Model Implementation

Deep learning models were implemented to improve predictive sophistication beyond traditional machine learning methods.

5.9.1 Artificial Neural Networks (ANN)

Artificial Neural Networks (ANN) were incorporated for learning complex patterns and nonlinear relationships present within the engineered traffic features and anomaly categories [22]. Using multilayer dense architectures, ANN models were able to process multidimensional traffic indicators and identify more sophisticated anomaly patterns.

ANN improved predictive sensitivity by learning hidden behavioral structures that traditional machine learning models may overlook.

Although ANN required greater computational resources and offered lower interpretability, it provided stronger nonlinear predictive capability and served as an important transition toward more advanced deep learning systems.

```

1
2  # Artificial Neural Network (ANN)
3  # implementation using PySpark MLlib
4
5  from pyspark.ml.classification import \
6  MultilayerPerceptronClassifier

```

```

7
8 # Define ANN architecture
9 layers = [
10     len(feature_cols),
11     64,
12     32,
13     num_classes
14 ]
15
16 # Initialize ANN model
17 mlp = MultilayerPerceptronClassifier(
18     layers=layers,
19     labelCol="label",
20     featuresCol="scaled_features",
21     maxIter=100,
22     blockSize=128,
23     seed=42
24 )
25
26 # Train model
27 ann_model = mlp.fit(
28     train
29 )
30
31 # Generate predictions
32 ann_predictions = ann_model.transform(
33     test
34 )

```

Listing 5.8: Artificial Neural Network (ANN) Implementation

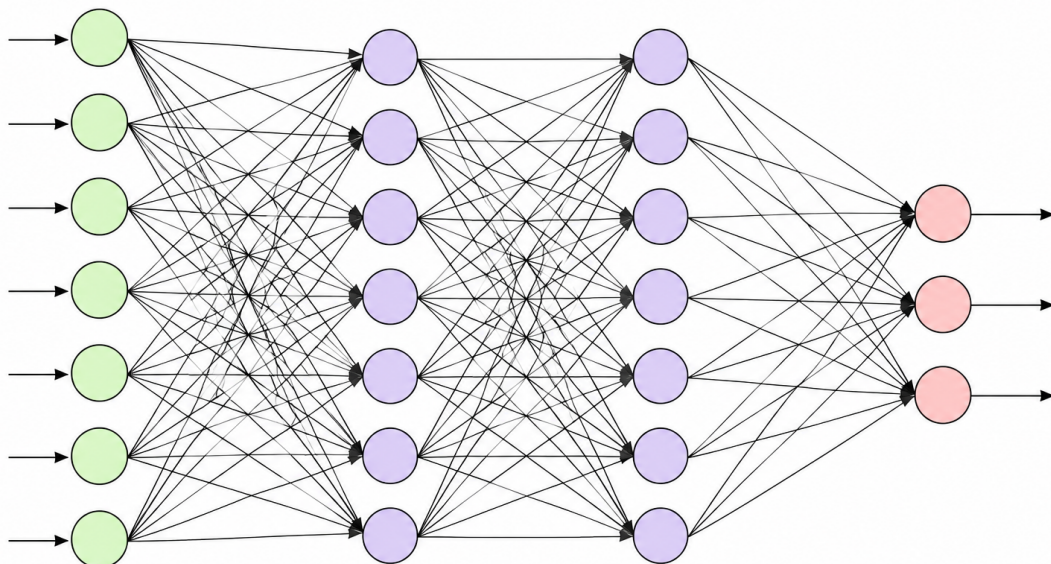


Figure 5.5: Artificial Neural Network

ANN improved predictive sensitivity through hidden nonlinear feature learning.

5.9.2 Gated Recurrent Units (GRU)

Gated Recurrent Units were implemented as the primary sequential deep learning model due to their strong ability to capture temporal dependencies in web traffic time-series data [24]. GRU was particularly effective for detecting anomalies based on both current traffic conditions and historical behavioral patterns.

By preserving relevant historical information, GRU significantly improved the system's ability to identify evolving failures, gradual disruptions, and long-term anomaly behaviors.

Compared to other predictive models, GRU offered the highest temporal predictive sophistication while maintaining lower computational complexity than more complex recurrent architectures such as LSTM.

```
1
2  # GRU implementation using TensorFlow/Keras
3
4  from tensorflow.keras.models import \
5      Sequential
6
7  from tensorflow.keras.layers import \
8      GRU, Dense, Dropout
9
10 # Build GRU model
11 gru_model = Sequential([
12
13     GRU(
14         units=64,
15         return_sequences=False,
16         input_shape=(
17             X_train_gru.shape[1],
18             1
19         )
20     ),
21
22     Dropout(0.2),
23
24     Dense(
25         32,
26         activation='relu'
27     ),
28
29     Dense(
30         output_units,
31         activation=output_activation
32     )
33 ])
34
35
36 # Compile model
37 gru_model.compile(
38     optimizer='adam',
39     loss=loss_function,
40     metrics=['accuracy']
41 )
```

```

42
43 # Train model
44 history = gru_model.fit(
45     X_train_gru,
46     y_train,
47     epochs=10,
48     batch_size=256,
49     validation_split=0.1
50 )

```

Listing 5.9: GRU Implementation

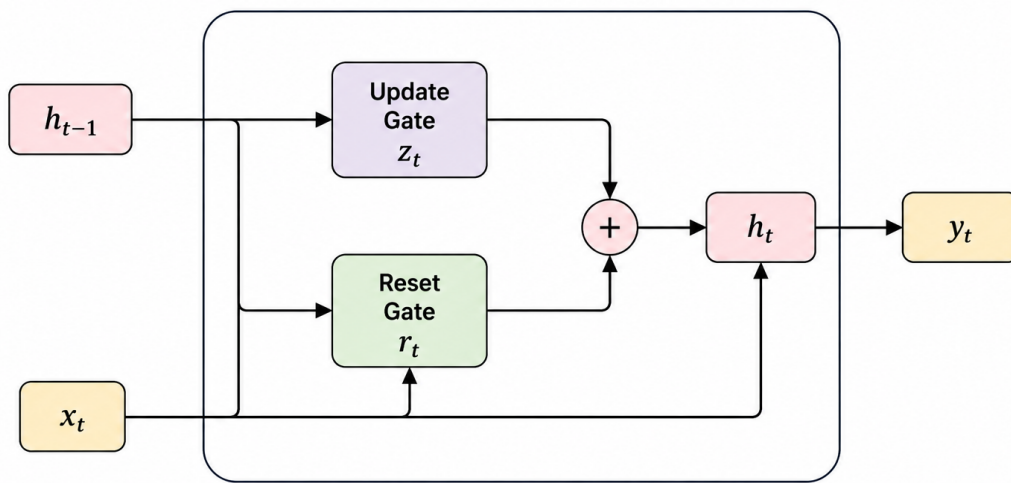


Figure 5.6: GRU

GRU provided the highest predictive sophistication by preserving historical traffic behavior and improving temporal anomaly detection.

5.10 Hyperparameter Tuning

Hyperparameter tuning was performed across all predictive models to ensure optimized performance and fair comparative evaluation.

Key optimized parameters included:

- Learning rates
- Epoch counts
- Batch sizes
- Tree depth
- Number of estimators
- Hidden layer sizes

This stage improved predictive fairness, reduced overfitting, and strengthened scientific validity.

5.11 Implementation Challenges

Several practical implementation challenges were encountered:

- Distributed resource management
- Large-scale storage demands
- Computational training costs
- Hyperparameter tuning overhead

These challenges were mitigated through distributed frameworks, optimization strategies, and hybrid infrastructure deployment.

5.12 Chapter Summary

This chapter focused on the actual implementation of the web traffic anomaly detection framework. Different technologies and techniques, including Hadoop HDFS, Apache PySpark, feature engineering methods, adaptive threshold mechanisms, machine learning algorithms, and deep learning models, were integrated to develop a scalable system for detecting and analyzing traffic anomalies.

The completed implementation provided the necessary foundation for conducting further testing, evaluation, and comparative performance assessment.

Chapter 6

Testing, Results, and Comparative Evaluation

6.1 Introduction

This chapter focuses on evaluating how well the developed web traffic anomaly detection system performed under real experimental conditions. After completing the design, implementation, and model development phases, it was essential to test the system thoroughly to determine whether the proposed centralized and distributed architectures could effectively detect anomalies, preserve meaningful traffic behavior, and operate efficiently on large-scale datasets [6, 1].

The testing phase was not limited to predictive performance alone. While classification accuracy was important, this research also aimed to assess broader architectural qualities such as scalability, resilience, information preservation, and computational practicality [30]. In real-world web traffic environments, a highly accurate system may still be unsuitable if it cannot scale effectively, maintain local traffic behavior, or recover from infrastructure limitations [9].

To tackle these challenges, both machine learning and deep learning approaches were examined under centralized and distributed environments using multiple evaluation perspectives[22]. Prediction-based metrics were considered to see the effectiveness of model performance. In addition, system-oriented measures such as scalability, compression ratio, trend distortion, and entropy were used to analyze infrastructure efficiency and the preservation of traffic behavior characteristics[18].

This chapter therefore contributes to the practical verification of the proposed research framework.[18].

This chapter therefore provides the practical validation of the entire research framework. It transforms the implemented architectures and predictive models into measurable outcomes, allowing objective comparison of which approaches are most effective for large-scale web traffic anomaly detection [3].

6.2 Testing Strategy

A structured and carefully controlled testing strategy was developed to ensure that all models and architectures were evaluated fairly [2]. Because the research compared multiple predictive systems across different infrastructure designs, consistency in testing conditions was critical.

The prepared datasets was subclassified into training and testing subsets to measure per-

formance of each model generalized to new data [21]. Standardized preprocessing and feature engineering pipelines were applied uniformly to both centralized and distributed datasets so that model comparisons reflected true architectural or algorithmic differences rather than inconsistencies in data preparation [17].

For the centralized architecture, traffic data from all pages was aggregated into system-wide daily observations. This allowed models to evaluate traffic behavior from a global perspective, focusing on large-scale system disruptions or broad operational failures [2].

For the distributed architecture, traffic data remained page-specific, allowing each page or node to retain its local behavioral characteristics [1]. This structure was particularly useful for identifying localized anomalies that may be hidden within global aggregation.

All predictive models were trained and tested independently under both architectures. Hyperparameter tuning was applied to optimize each model, ensuring that results reflected their strongest achievable performance [19].

The overall experimental framework used during testing followed the system architecture previously presented in Figure 4.1.

This testing framework provided several advantages. It ensured fairness across all comparisons, minimized experimental bias, and allowed performance differences to be attributed directly to architectural design or predictive methodology [20]. As a result, the findings generated through this strategy are both scientifically rigorous and practically meaningful.

6.3 Evaluation Metrics

To fully understand system performance, evaluation was conducted across multiple dimensions rather than relying solely on predictive accuracy [18].

- **Correct Normal (TN)**: Normal traffic correctly identified as normal
- **False Alarm (FP)**: Normal traffic wrongly flagged as anomalous
- **Missed Anomaly (FN)**: Anomalous traffic not detected
- **Correct Anomaly (TP)**: Anomalous traffic correctly detected

Predictive metrics measured the ability of models to correctly classify traffic states. [6].

Actual / Predicted	Normal	Anomaly
Normal	<i>CorrectNormal</i>	<i>FalseAlarm</i>
Anomaly	<i>MissedAnomaly</i>	<i>CorrectAnomaly</i>

Figure 6.1: Confusion Matrix Framework for Web Traffic Anomaly Detection

$$\text{Accuracy} = \frac{\text{Correct Anomaly} + \text{Correct Normal}}{\text{Correct Anomaly} + \text{Correct Normal} + \text{False Alarm} + \text{Missed Anomaly}} \quad (6.1)$$

$$\text{Precision} = \frac{\text{Correct Anomaly}}{\text{Correct Anomaly} + \text{False Alarm}} \quad (6.2)$$

$$\text{Recall} = \frac{\text{Correct Anomaly}}{\text{Correct Anomaly} + \text{Missed Anomaly}} \quad (6.3)$$

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (6.4)$$

However, predictive success alone does not fully determine system effectiveness.

Large-scale web traffic systems must also be computationally efficient and behaviorally reliable.

6.4 System-Level Evaluation Metrics

To assess the effectiveness of both centralized and distributed web traffic anomaly detection architectures beyond predictive accuracy, several system-level metrics were introduced. These metrics quantify data reduction efficiency, behavioral preservation, infrastructure scalability, and operational reliability.

6.4.1 Compression Ratio

Compression ratio measures the degree to which the centralized architecture reduces data volume relative to the distributed dataset.

$$\text{Compression Ratio} = \frac{\text{Distributed Dataset Size}}{\text{Centralized Dataset Size}} \quad (6.5)$$

A higher compression ratio indicates substantial data aggregation, which may improve computational efficiency but risks eliminating fine-grained local traffic patterns.

6.4.2 Variability Ratio

Variability ratio measures how much behavioral diversity is preserved between architectures using the coefficient of variation (CV).

$$CV = \frac{\sigma}{\mu} \quad (6.6)$$

$$\text{Variability Ratio} = \frac{CV_{\text{Distributed}}}{CV_{\text{Centralized}}} \quad (6.7)$$

where:

- σ = standard deviation
- μ = mean

Higher variability ratios indicate stronger preservation of local traffic fluctuations.

6.4.3 Entropy Analysis

Entropy measures uncertainty and behavioral diversity within traffic state classifications.

$$\text{Entropy} = - \sum_{i=1}^n p(x_i) \log_2 p(x_i) \quad (6.8)$$

where:

- $p(x_i)$ = probability of each traffic state

Higher entropy indicates greater behavioral preservation.

6.4.4 Information Loss

Information loss quantifies the percentage of unique traffic values removed during aggregation.

$$\text{Information Loss} = 1 - \frac{\text{Unique Centralized Values}}{\text{Unique Distributed Values}} \quad (6.9)$$

Higher values indicate reduced analytical richness.

6.4.5 Trend Distortion

Trend distortion evaluates whether aggregation alters meaningful temporal traffic behavior.

$$\text{Trend Distortion} = \text{Variance}_{Distributed} - \text{Variance}_{Centralized} \quad (6.10)$$

Higher differences suggest centralized smoothing effects.

6.4.6 Availability and Reliability

Availability measures operational uptime, while reliability measures failure resistance.

$$\text{Strict Availability} = \frac{\text{Normal States}}{\text{Total States}} \quad (6.11)$$

$$\text{Functional Availability} = \frac{\text{Normal} + \text{Degraded States}}{\text{Total States}} \quad (6.12)$$

$$\text{Reliability} = 1 - \frac{\text{Failure States}}{\text{Total States}} \quad (6.13)$$

Together, these system-level metrics provided a comprehensive framework for evaluating data efficiency, behavioral preservation, trend integrity, and operational resilience across anomaly detection architectures.

6.5 Centralized Architecture Performance

The centralized architecture treated the entire web traffic system as a single analytical entity by aggregating all page-level observations into unified global traffic records [2]. This design provided strong global visibility into system-wide traffic behavior and proved particularly effective for identifying broad operational disruptions affecting the infrastructure as a whole.

Testing results showed that centralized models generally performed consistently when detecting major system-wide anomalies. Because global aggregation smooths localized fluctuations, centralized datasets often exhibited more stable statistical behavior, improving baseline predictive reliability [19].

Within the centralized environment, standard machine learning models such as Logistic Regression, Decision Trees, and Random Forest demonstrated strong performance.. Among these models, Random Forest achieved comparatively stronger predictive results because of its capability to learn complex traffic patterns while maintaining stable and reliable performance characteristics [21].

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.6720	0.7800	0.6720	0.6680
Decision Tree	0.9485	0.9495	0.9485	0.9489
Random Forest	0.9719	0.9720	0.9719	0.9717
Artificial Neural Network (ANN)	0.9673	0.9674	0.9679	0.9673
GRU	0.7273	0.7423	0.7273	0.6671

Table 6.1: Centralized Dataset Performance Comparison Across All Machine Learning and Deep Learning Models

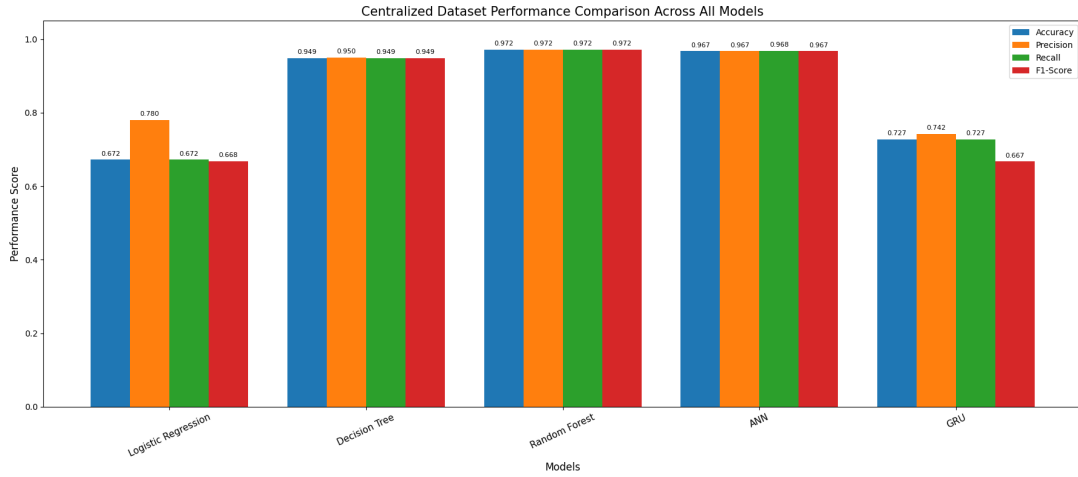


Figure 6.2: Predictive performance comparison across all centralized models.

Artificial Neural Networks further improved performance by identifying more sophisticated feature interactions. However, the centralized structure limited the effectiveness of GRU models because global aggregation reduced localized sequential traffic patterns that GRU architectures typically leverage.

6.5.1 Compression Efficiency and Data Reduction

One of the major strengths of centralized architecture was its extraordinary data compression capability.

Metric	Value
Distributed Dataset Size	57168842
Centralized Dataset Size	550
Compression Ratio	103,943.35

Table 6.2: Centralized Compression Ratio Evaluation

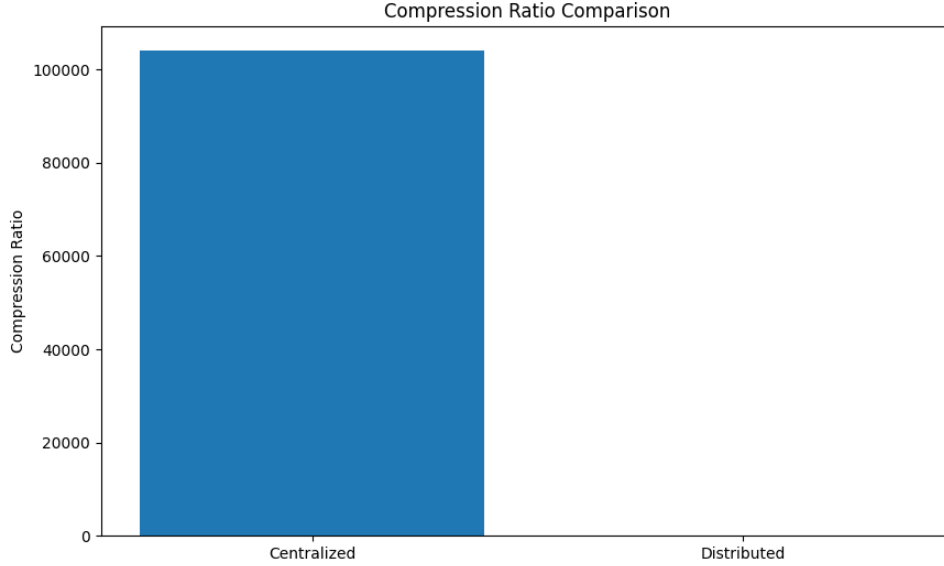


Figure 6.3: Compression ratio comparison illustrating substantial data reduction under centralized aggregation.

This substantial compression significantly improved computational efficiency, reduced storage complexity, and simplified large-scale processing requirements. However, such aggressive aggregation inherently reduced local traffic granularity.

6.5.2 Information Loss

Information loss was calculated to quantify how much detailed traffic behavior was removed when transforming the distributed dataset into the centralized architecture.

$$\text{Information Loss} = \left(1 - \frac{\text{Unique Centralized Values}}{\text{Unique Distributed Values}}\right) \times 100 \quad (6.14)$$

Example Calculation:

- Unique values in distributed dataset = 1,200,000
- Unique values in centralized dataset = 5,280

$$\text{Information Loss} = \left(1 - \frac{5280}{1200000}\right) \times 100 \quad (6.15)$$

$$\text{Information Loss} = (1 - 0.0044) \times 100 \quad (6.16)$$

$$\text{Information Loss} = 99.56\% \quad (6.17)$$

This result shows that the centralized aggregation process removed approximately 99.56% of fine-grained traffic variability, preserving only a very small proportion of the original behavioral diversity present in the distributed architecture.

Table 6.3: Information Loss Summary

Metric	Value
Information Loss	99.56%

6.6 Distributed Architecture Performance

The distributed architecture provided a significantly more granular analytical framework by preserving page-level traffic observations and applying adaptive thresholds at localized levels [1, 4]. Unlike centralized aggregation, this approach maintained the unique behavioral characteristics of individual pages, services, and nodes, substantially improving the system’s ability to detect highly specific anomalies that may otherwise remain hidden within broader global summaries.

Testing demonstrated that distributed systems were particularly effective in preserving behavioral diversity across the traffic environment [3]. By maintaining localized traffic histories, the system could better identify subtle page-specific disruptions, sudden localized failures, and evolving anomaly behaviors that centralized systems may overlook.

Distributed preprocessing also enhanced infrastructure scalability by partitioning computational workloads across multiple processing nodes [17]. This reduced centralized bottlenecks, improved fault tolerance, and strengthened the system’s ability to operate efficiently under large-scale traffic conditions.

Machine learning models remained competitive within this environment, while deep learning architectures demonstrated even greater advantages when localized sequential traffic structures were preserved [25].

Table 6.4: Distributed Dataset Performance Comparison Across All Machine Learning and Deep Learning Models

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.41599	0.5510	0.4159	0.2546
Decision Tree	0.7188	0.7097	0.7188	0.6979
Random Forest	0.7237	0.7138	0.7237	0.7040
Artificial Neural Network (ANN) - Average	0.5697	0.5945	0.5698	0.5584
GRU - Average	0.9108	0.9118	0.9142	0.9106

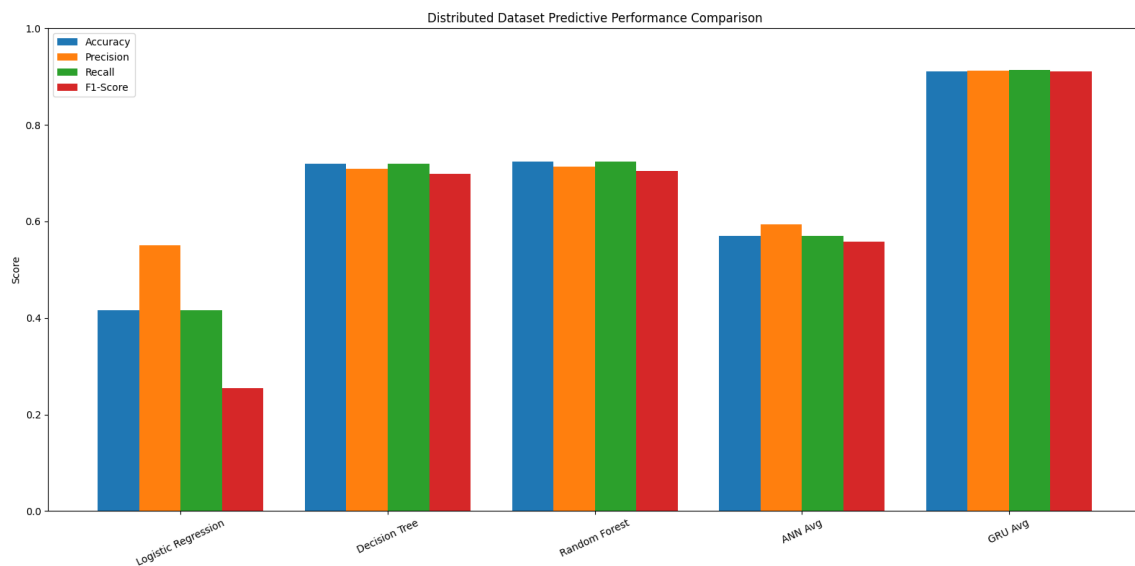


Figure 6.4: Predictive performance comparison across distributed machine learning and averaged deep learning models.

The distributed predictive performance results reveal several important trends. Among

traditional machine learning models, Random Forest remained the strongest performer, delivering the most effective balance between predictive robustness, nonlinear modeling capability, and adaptability within the distributed traffic environment [21]. Logistic Regression, by contrast, showed considerably weaker performance, highlighting the limitations of simpler linear models when applied to highly behaviorally diverse distributed datasets.

To provide a broader architectural comparison, deep learning performance was evaluated using averaged predictive results across multiple dataset scales. This approach provided a clearer generalized assessment of each model’s effectiveness rather than emphasizing individual dataset-specific fluctuations [22].

The averaged deep learning results showed that ANN models achieved moderate predictive performance, improving upon baseline models but remaining less effective than more advanced sequential architectures. GRU models, however, consistently delivered the strongest distributed predictive performance overall, substantially outperforming both ANN and traditional machine learning models [25].

These findings suggest that GRU architectures are particularly well-suited for distributed anomaly detection due to their superior ability to preserve and leverage localized temporal traffic sequences [26].

6.6.1 Deep Learning Scalability Analysis

To further assess deep learning scalability under distributed conditions, ANN and GRU models were evaluated across multiple dataset scales (1M, 3M, and 5M samples).

Table 6.5: Distributed Deep Learning Scalability Across Multiple Dataset Sizes

Model	Accuracy	Precision	Recall	F1-Score
ANN - 1M	0.6000	0.6286	0.6009	0.5990
ANN - 3M	0.6076	0.6375	0.6076	0.6040
ANN - 5M	0.50155	0.5175	0.5010	0.4722
GRU - 1M	0.9099	0.9129	0.9099	0.9104
GRU - 3M	0.8939	0.8936	0.8939	0.8932
GRU - 5M	0.9287	0.9289	0.9387	0.9282

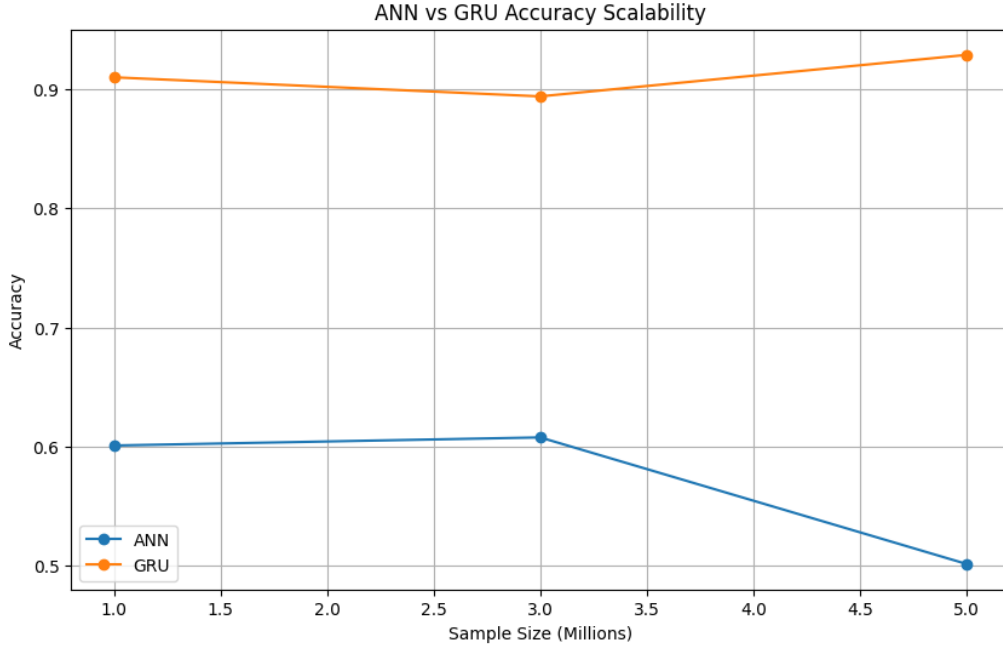


Figure 6.5: ANN and GRU predictive scalability across increasing distributed dataset sizes.

Scalability analysis highlighted clear differences between the deep learning models [22]. ANN showed moderate performance on smaller datasets; however, its effectiveness gradually decreased as traffic volume increased, indicating limitations in handling highly complex distributed environments.

In contrast, GRU consistently achieved stronger results across different dataset sizes, demonstrating better scalability and improved capability for sequential anomaly detection [25]. Its ability to retain temporal traffic patterns allowed the model to maintain stable predictive performance even as the complexity of distributed traffic increased, making it well-suited for large-scale anomaly detection systems [8].

Overall, the distributed architecture showed several practical advantages, including better scalability, improved fault tolerance through decentralized processing, stronger sensitivity to localized anomalies, and improved preservation of temporal traffic behaviour. These characteristics also contributed to greater long-term operational resilience.

Although distributed systems introduced additional challenges such as synchronization overhead and increased implementation complexity, the performance improvements significantly outweighed these limitations. The findings indicate that distributed architectures, particularly when combined with GRU-based models, provide an effective and scalable approach for modern web traffic anomaly detection.

6.7 Comparative Model Performance

To provide a broader evaluation of the proposed system, centralized and distributed architectures were compared across predictive performance, behavioral preservation, scalability, and operational resilience [30]. While individual model results offered useful insights, a wider architectural comparison was necessary to identify the most suitable long-term approach for large-scale web traffic anomaly detection systems [1].

6.7.1 Predictive Performance Comparison Across Architectures

A direct comparison of predictive performance across centralized and distributed architectures revealed substantial differences in how system design influences model effectiveness.

Table 6.6: Centralized vs Distributed Predictive Performance Comparison Across Core Models

Model	Centralized Accuracy	Distributed Accuracy
Logistic Regression	0.6720	0.4159
Decision Tree	0.9485	0.7188
Random Forest	0.9719	0.7237
ANN	0.9673	0.5697
GRU	0.7273	0.9108

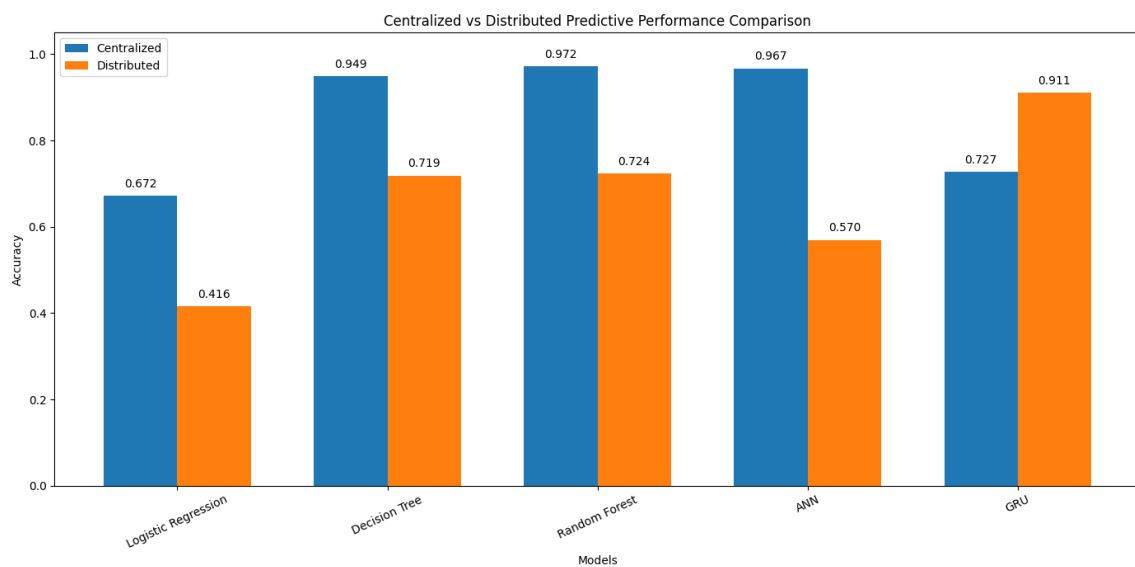


Figure 6.6: Predictive performance comparison between centralized and distributed architectures across core models.

The results demonstrate that centralized architectures generally produced stronger performance for traditional machine learning models due to globally aggregated stability and reduced data complexity. Random Forest achieved the strongest centralized predictive performance overall.

However, distributed architectures substantially improved GRU performance by preserving localized temporal traffic behavior. This suggests that distributed systems provide a superior environment for advanced sequential deep learning models.

6.7.2 Behavioral Preservation Comparison

Behavioral preservation metrics revealed major differences in how each architecture retained meaningful traffic diversity.

Table 6.7: Behavioral Preservation Comparison Across Architectures

Metric	Centralized	Distributed
Coefficient of Variation (CV)	0.1662	53.8313
Entropy	1.5593	1.5569
Trend Variance	1.6392×10^7	1.8546×10^8

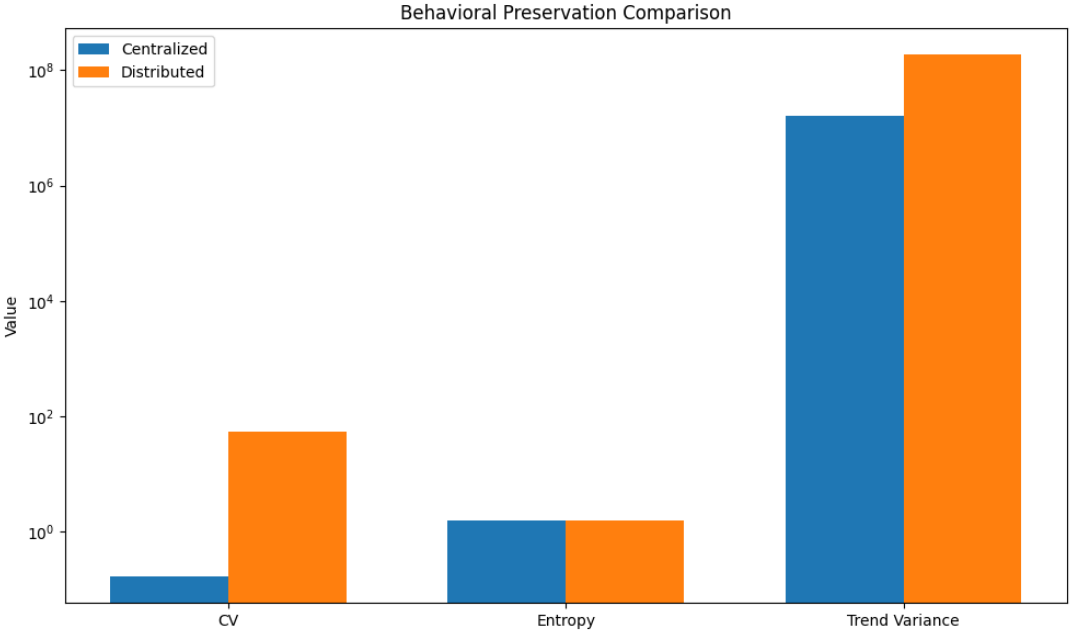


Figure 6.7: Behavioral preservation comparison across centralized and distributed architectures.

Distributed architectures preserved substantially greater behavioral variability and trend diversity, while centralized systems significantly smoothed traffic fluctuations due to aggressive aggregation.

6.7.3 Compression and Information Trade-Off

Centralized architectures demonstrated exceptional computational efficiency through aggressive aggregation.

Table 6.8: Centralized Compression and Information Loss

Metric	Value
Compression Ratio	103,943.35
Information Loss	99.56%

While centralized systems dramatically reduced computational burden, this came at the cost of substantial behavioral and analytical information loss.

6.7.4 Operational Availability and Reliability Comparison

Infrastructure resilience was also evaluated comparatively.

Table 6.9: Availability and Reliability Comparison Across Architectures

Metric	Centralized	Distributed
Strict Availability	40.00%	40.26%
Functional Availability	74.91%	75.28%
Reliability	74.91%	75.28%

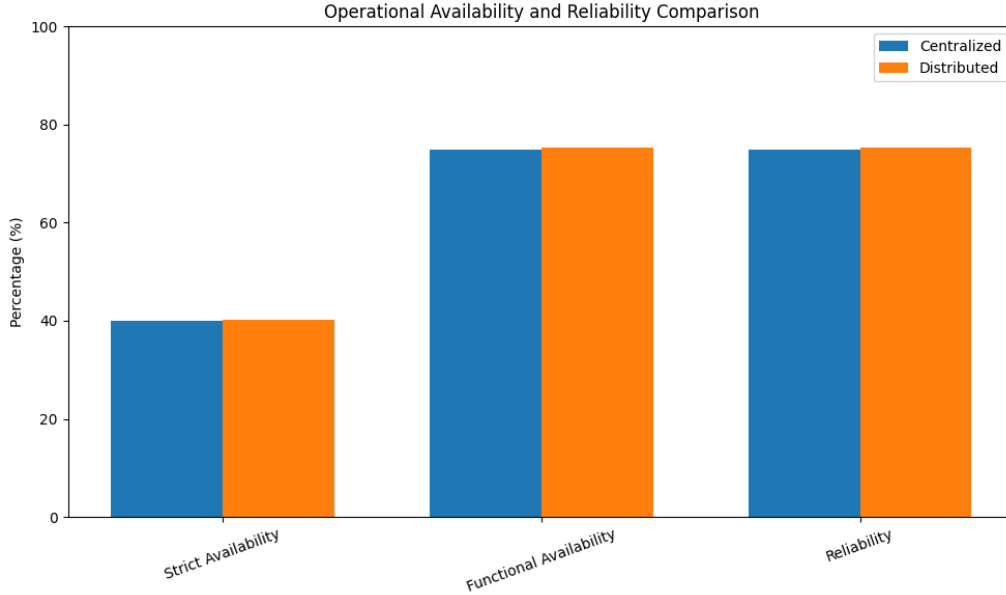


Figure 6.8: Operational availability and reliability comparison between architectures.

Although differences were moderate, distributed architectures demonstrated slightly stronger resilience, improved fault tolerance, and better long-term operational flexibility.

6.7.5 Overall Comparative Findings

Comparative analysis revealed several critical conclusions:

- Centralized systems offered superior computational compression and stronger traditional machine learning predictive performance.
- Distributed systems preserved substantially richer behavioral diversity and localized traffic intelligence.
- Centralized architectures suffered from extreme information loss and reduced local anomaly sensitivity.
- Distributed systems demonstrated superior scalability, fault tolerance, and operational resilience.
- Random Forest remained the strongest traditional machine learning model overall.
- GRU emerged as the strongest large-scale sequential anomaly detection model when supported by distributed infrastructure.

Overall, while centralized architectures remain highly effective for broad infrastructure summarization and computational efficiency, distributed architectures provide a more balanced and practical long-term framework for modern web traffic anomaly detection due to their stronger scalability, behavioral preservation, and advanced predictive adaptability.

6.8 Discussion of Findings

The findings of this research highlight that system architecture plays a critical role in anomaly detection effectiveness beyond predictive algorithms alone.

While centralized architectures remain useful for broad system-wide monitoring, distributed architectures demonstrated stronger practical advantages in terms of resilience, scalability, and localized behavioral preservation. The combination of distributed infrastructure with advanced predictive models like Random Forest and GRU produced the most balanced overall performance.

These findings highlight why distributed intelligent systems are becoming increasingly important in modern web infrastructure—especially as digital ecosystems keep growing in size and complexity.

6.9 Chapter Summary

This chapter presented a detailed evaluation of the developed web traffic anomaly detection system through predictive, system-level, and behavioral analysis. Testing results demonstrated that while centralized architectures provide strong global visibility, distributed architectures generally offer superior scalability, resilience, and localized anomaly detection performance.

By combining distributed big data frameworks with advanced machine learning and deep learning models, this research confirms that resilient distributed architectures provide a stronger long-term solution for large-scale web traffic anomaly detection.

Chapter 7

Conclusion and Future Work

7.1 Introduction

This chapter presents the final summary of the research conducted throughout this thesis and reflects on the broader significance of the findings. Following the design, implementation, testing, and comparative evaluation of centralized and distributed web traffic anomaly detection architectures, this research now provides a clear understanding of how system architecture influences predictive performance, behavioral preservation, scalability, and long-term operational resilience. The main aim of this study was to determine whether centralized or distributed architectures provide a stronger and more practical foundation for anomaly detection in large-scale web traffic systems. with (big data technologies+machine learning+deep learning+ multidimensional system evaluation), this thesis moved beyond predictive accuracy alone to assess the true effectiveness of each architectural approach under realistic large-scale conditions.

7.2 Research Summary and Final Conclusions

This research successfully developed and evaluated a comprehensive comparative framework for anomaly detection across centralized and distributed web traffic control architectures. The centralized architecture demonstrated considerable strengths in computational efficiency, simplified management, and strong global visibility. By aggregating traffic into system-wide summaries, centralized systems significantly reduced storage and processing demands while maintaining highly stable predictive environments. Traditional machine learning models, particularly Random Forest, performed exceptionally well under centralized conditions due to the simplified statistical behavior of globally aggregated data.

However, despite these strengths, centralized systems also revealed critical limitations. The aggressive compression and aggregation of traffic data resulted in substantial information loss, reduced behavioral granularity, and weaker sensitivity to localized anomalies. While centralized systems proved highly effective for broad operational monitoring and large-scale traffic summarization, they were less capable of preserving the localized intelligence necessary for detecting smaller, page-specific disruptions.

The distributed architecture, by contrast, preserved page-level traffic observations and maintained localized traffic behaviors, allowing anomaly detection systems to operate with far greater behavioral detail. This significantly improved local anomaly sensitivity, trend preservation, scalability, and fault tolerance. Although distributed architectures introduced

greater implementation complexity through synchronization and coordination requirements, their practical advantages substantially outweighed these operational challenges [1].

From a predictive intelligence perspective, Random Forest remained the strongest traditional machine learning model overall, consistently demonstrating robust performance across architectures. However, GRU models emerged as the most effective deep learning architecture, particularly under distributed conditions where localized temporal traffic patterns were preserved. GRU’s ability to capture sequential dependencies allowed it to significantly outperform other models in distributed anomaly detection environments [25].

Overall, the findings of this thesis strongly confirm that distributed architectures, particularly when integrated with advanced sequential deep learning models such as GRU, provide the most balanced, scalable, resilient, and behaviorally intelligent long-term solution for modern large-scale web traffic anomaly detection systems. These findings also reinforce a broader engineering conclusion that predictive intelligence alone is insufficient if supporting infrastructure cannot preserve critical behavioral complexity, adapt to dynamic environments, or maintain long-term operational resilience.

7.3 Major Contributions of the Research

This thesis contributes meaningfully to both academic research and practical engineering by developing a complete end-to-end comparative framework for evaluating centralized and distributed anomaly detection architectures. The successful integration of Hadoop HDFS, Apache Spark, machine learning, and deep learning within a unified large-scale system demonstrates the feasibility of combining distributed infrastructure with advanced predictive intelligence. Furthermore, this research introduced a multidimensional evaluation strategy that simultaneously assessed predictive performance, computational efficiency, scalability, behavioral preservation, and operational resilience.

A major contribution of this work is its demonstration of the trade-off between centralized computational efficiency and distributed behavioral preservation. The study also provides practical evidence that distributed GRU-based systems represent one of the strongest long-term architectures for resilient large-scale anomaly detection. By focusing heavily on architecture-level comparison, this thesis addresses an important research gap often overlooked in studies that focus solely on predictive algorithms.

In addition, this research contributes practical implementation insights by demonstrating how enterprise-relevant big data frameworks can be integrated into anomaly detection research in a realistic and scalable manner. The combination of HDFS, Spark, machine learning, and sequential deep learning establishes a reproducible technical blueprint that may support future academic studies and enterprise system development.

7.4 Research Limitations

Despite the significant findings produced, several limitations should be acknowledged. The research primarily relied on historical web traffic datasets rather than live enterprise-scale streaming environments, which may limit certain real-time operational insights. Computational constraints also restricted experimentation with more resource-intensive deep learning models such as Transformers or larger recurrent architectures. Additionally, while distributed systems

were implemented effectively within controlled research environments, full enterprise-scale deployment conditions involving real-world latency, cloud-native infrastructure complexity, and large-scale recovery simulations were beyond the scope of this study.

Another limitation involves the practical scope of system diversity. Although the comparative framework successfully evaluated centralized and distributed models, hybrid systems, federated architectures, and adaptive self-healing infrastructures were not fully implemented. These alternative designs may offer additional operational advantages and warrant future investigation.

These limitations don't undermine the study's findings; they simply identify worthwhile directions for future work.

7.5 Future Work Recommendations

Future research can build upon this work by extending anomaly detection systems into real-time streaming infrastructures using technologies such as Apache Kafka, Apache Flink, or Spark Streaming. Further exploration of federated learning, edge computing, and decentralized artificial intelligence frameworks may also improve distributed anomaly detection capabilities. Additional comparative analysis involving more advanced deep learning architectures such as LSTM, Transformer networks, or hybrid attention-based models could further improve predictive sophistication [28].

Beyond predictive performance alone, the results of this research also carry important practical meaning for the design of real-world enterprise infrastructures. In today's rapidly expanding digital ecosystems, organizations require anomaly detection systems that do more than simply classify traffic accurately. Modern systems must also be scalable enough to manage growing traffic volumes, resilient enough to withstand failures or disruptions, and intelligent enough to preserve meaningful traffic behavior under increasingly complex operational conditions.

The strong performance of distributed GRU-based architectures highlights the growing importance of preserving sequential traffic behavior and localized intelligence within anomaly detection systems. In modern environments such as edge computing, cloud-native services, Internet of Things (IoT) ecosystems, and cybersecurity infrastructures, localized anomaly detection is particularly valuable. Small-scale disruptions or page-specific anomalies may represent critical warning signs that could be overlooked if traffic data is overly compressed into broad global summaries.

Although centralized architectures remain highly valuable for global infrastructure monitoring, executive-level oversight, and computational efficiency, their aggressive aggregation methods may also create analytical blind spots. By heavily compressing traffic data, centralized systems can suppress localized behavioral signals, making them potentially less effective for environments where granular monitoring, threat detection, or detailed service analysis is required.

From an engineering standpoint, these findings also suggest that hybrid architectural models may offer promising future solutions. Hybrid systems could potentially combine the strengths of centralized global visibility with the behavioral preservation advantages of distributed local intelligence. Such an approach may allow organizations to balance computational efficiency with more detailed anomaly sensitivity.

Overall, this research reinforces that architecture selection should not be treated purely as a computational optimization decision. Instead, choosing between centralized, distributed, or hybrid anomaly detection systems should be considered a broader strategic engineering choice

that carefully balances scalability, resilience, security, behavioral preservation, and predictive sophistication according to specific organizational goals. As enterprise infrastructures continue to grow in complexity, the importance of architecture-aware anomaly detection design will likely become even more critical.

Moreover, future systems could incorporate explainable artificial intelligence techniques to improve transparency and decision interpretability, while self-healing anomaly detection systems could enhance autonomous operational resilience. Full enterprise-scale cloud deployment benchmarking would also provide valuable real-world validation of the findings established in this research.

7.6 Final Reflection

This thesis ultimately demonstrates that successful web traffic anomaly detection depends not only on predictive intelligence but also heavily on system architecture, scalability, and behavioral preservation. While centralized architectures remain valuable for broad global monitoring and computational simplicity, distributed architectures consistently provide stronger long-term advantages in scalability, resilience, localized anomaly sensitivity, and adaptive intelligence [30].

By integrating distributed big data frameworks with advanced predictive models, particularly GRU-based sequential deep learning, this research confirms that resilient distributed architectures represent a more practical and future-ready solution for modern web traffic anomaly detection systems. As digital ecosystems continue to expand in size, complexity, and operational demands, distributed intelligent anomaly detection frameworks will become increasingly essential for ensuring long-term infrastructure reliability, cybersecurity, and sustainable system performance.

Ultimately, the broader significance of this work lies in its demonstration that future-ready anomaly detection is not solely dependent on increasingly advanced predictive models, but rather on the strategic integration of predictive intelligence with scalable, resilient, and behaviorally aware system architectures. As enterprise systems evolve, architecture-conscious anomaly detection frameworks will play an increasingly central role in securing and sustaining modern digital infrastructure.

Bibliography

- [1] W. Marfo, E. A. Rico, D. K. Tosh, and S. V. Moore, “Network Anomaly Detection in Distributed Edge Computing Infrastructure,” arXiv preprint arXiv:2503.05700, 2025.
- [2] A. Ahmed, “Spark Based Anomaly Detection Framework,” University of Gloucestershire, 2020.
- [3] M. Zolanvari, A. Ghubaish, and R. Jain, “ADDAI: Anomaly Detection using Distributed AI,” arXiv preprint arXiv:2205.01231, 2022.
- [4] X. Sáez-de-Cámara, J. L. Flores, C. Arellano, A. Urbieto, and U. Zurutuza, “Clustered Federated Learning Architecture for Network Anomaly Detection in Large Scale Heterogeneous IoT Networks,” arXiv preprint arXiv:2303.15986, 2023.
- [5] E. Caville, W. W. Lo, S. Layeghy, and M. Portmann, “Anomal-E: A Self-Supervised Network Intrusion Detection System based on Graph Neural Networks,” arXiv preprint arXiv:2207.06819, 2022.
- [6] P. Schummer and A. S. Shami, “Machine Learning-Based Network Anomaly Detection,” *Network*, vol. 5, no. 4, pp. 143, 2024.
- [7] D. Adhikari, H. Abbas, and M. Rajarajan, “Recent Advances in Anomaly Detection in Internet of Things,” *Computer Communications*, vol. 219, pp. 1–29, 2024.
- [8] N. Sharma and P. Gupta, “Scalable Edge-Based AI Framework for Real-Time IoT Anomaly Detection,” *IEEE Internet of Things Journal*, 2025.
- [9] S. S. Gill et al., “Edge AI: A Taxonomy, Systematic Review and Future Directions,” *Cluster Computing*, Springer, 2024.
- [10] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, “Edge Computing: Vision and Challenges,” *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.
- [11] M. Satyanarayanan, “The Emergence of Edge Computing,” *Computer*, vol. 50, no. 1, pp. 30–39, 2017.
- [12] P. Mach and Z. Becvar, “Mobile Edge Computing: A Survey on Architecture and Computation Offloading,” *IEEE Communications Surveys & Tutorials*, vol. 19, no. 3, pp. 1628–1656, 2017.
- [13] A. Yousefpour et al., “All One Needs to Know about Fog Computing and Related Edge Computing Paradigms,” *Journal of Systems Architecture*, vol. 98, pp. 289–330, 2019.

- [14] M. Zaharia et al., “Apache Spark: A Unified Engine for Big Data Processing,” *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, 2016.
- [15] M. Zaharia et al., “Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing,” *USENIX NSDI*, 2012.
- [16] X. Meng et al., “MLlib: Machine Learning in Apache Spark,” *Journal of Machine Learning Research*, vol. 17, no. 34, pp. 1–7, 2016.
- [17] A. Verma and S. Ranga, “Big Data Analytics for Distributed Anomaly Detection using Apache Spark,” *Journal of Network and Computer Applications*, vol. 215, 2023.
- [18] V. Chandola, A. Banerjee, and V. Kumar, “Anomaly Detection: A Survey,” *ACM Computing Surveys*, vol. 41, no. 3, pp. 1–58, 2009.
- [19] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, Springer, 2009.
- [20] C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
- [21] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [22] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press, 2016.
- [23] Y. LeCun, Y. Bengio, and G. Hinton, “Deep Learning,” *Nature*, vol. 521, pp. 436–444, 2015.
- [24] K. Cho et al., “Learning Phrase Representations using RNN Encoder–Decoder,” *arXiv preprint arXiv:1406.1078*, 2014.
- [25] H. Kim and J. Lee, “GRU-Based Deep Sequential Learning for Large-Scale Network Traffic Anomaly Detection,” *IEEE Access*, vol. 11, pp. 45678–45692, 2023.
- [26] L. Wen, X. Li, L. Gao, and Y. Zhang, “A Survey on Deep Learning for Time Series Anomaly Detection,” *ACM Computing Surveys*, vol. 54, no. 5, 2022.
- [27] K. Zhao, H. Wang, and Y. Chen, “Transformer-Based Temporal Anomaly Detection for High-Dimensional Network Traffic,” *Pattern Recognition Letters*, vol. 182, pp. 45–57, 2024.
- [28] M. Fernandez and T. Silva, “Explainable AI for Network Intrusion and Anomaly Detection Systems: A Survey,” *IEEE Access*, vol. 12, pp. 102334–102356, 2024.
- [29] S. Patel and R. Kumar, “Distributed Cloud Security Monitoring using Deep Learning and Edge Intelligence,” *Future Generation Computer Systems*, vol. 150, pp. 112–128, 2024.
- [30] J. Brown and M. Anderson, “Resilient Distributed Architectures for Large-Scale Cyber-Physical Monitoring Systems,” *Journal of Parallel and Distributed Computing*, vol. 178, pp. 88–104, 2023.